MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

Sistem de detecție comportamentală pentru macOS folosind Apple Endpoint Security Framework

LUCRARE DE DISERTAȚIE

Absolvent masterand: **Dimitrie-Toma Furdui**

Coordonator științific: **Conf. Dr. Ing. Adrian Coleșa,**
Dr. Ing. Radu Marian Portase

Iulie, 2025

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

DECAN
Prof.Dr.Ing. Vlad MUREȘAN

DIRECTOR DEPARTAMENT
Prof.Dr.Ing. Rodica POTOLEA

Student masterand: **Dimitrie-Toma Furdui**

Sistem de detecție comportamentală pentru macOS folosind Apple Endpoint Security Framework

1. **Enunțul temei:** Folosirea Apple Endpoint Security Framework pentru a scrie un sistem de detecție comportamentală, capabil să identifice și să blocheze atacuri complexe ale aplicațiilor malware moderne, compatibil cu regulile Sigma și testat cu suita de teste Atomic Red Team
2. **Conținutul lucrării:** Pagina de prezentare, Introducere, Stadiul actual în domeniu, Prezentarea proiectului, Rezultate teoretice și experimentale, Concluzii, Bibliografie
3. **Locul documentării:** UTCN, Cluj-Napoca
4. **Consultanți:** Conf. Dr. Ing. Adrian Colesa, Dr. Ing. Radu Marian Portase
5. **Data emiterii temei:** 18 iulie 2023
6. **Data predării:** 11 iulie 2025

Student masterand: .....

Coordonator științific: .....

MINISTERUL EDUCAȚIEI ȘI CERCETĂRII

UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

Declarație pe proprie răspundere privind autenticitatea lucrării de disertație

Subsemnatul(a) Dimitrie-Toma Furdui, legitimat(ă) cu C.I. seria CJ nr. 1029591, CNP 5011018046196, autorul lucrării Sistem de detecție comportamentală pentru macOS folosind Apple Endpoint Security Framework, elaborată în vederea susținerii examenului de finalizare a studiilor de disertație la Facultatea de Automatică și Calculatoare, specializarea Securitatea Informațiilor și Sistemelor de Calcul, din cadrul Universității Tehnice din Cluj-Napoca, sesiunea Iulie a anului universitar 2024-2025, declar pe proprie răspundere că această lucrare este rezultatul propriei activități intelectuale, pe baza cercetărilor mele şi pe baza informațiilor obținute din surse care au fost citate în textul lucrării și în bibliografie.

Declar că această lucrare nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române și a convențiilor internaționale privind drepturile de autor.

Declar, de asemenea, că această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de disertație.

În cazul constatării ulterioare a unor declarații false, voi suporta sancțiunile administrative, respectiv, *anularea examenului de disertație*.

Sunt de acord ca, pe tot parcursul vieții, în cazul în care este necesar și se va dori verificarea autenticității lucrării mele să fiu identificat și verificat în baza datelor declarate de mine.

Data

11.07.2025

Dimitrie-Toma Furdui

.....

Semnătura

Cuprins

Capitolul 1 Introducere **1**
1.1 Relevanța subiectului lucrării 1
1.2 Motivație 2
1.3 Sisteme de detecție al comportamentului rău intenționat 3
1.3.1 Sistem de detecție comportamentală 3
1.3.2 Sistem de detecție bazat pe injecție în procese 4
1.4 Obiective 6
1.5 Contribuții 6
1.6 Structura tezei 7

Capitolul 2 Stadiul actual în domeniu **9**
2.1 Răspândirea aplicațiilor malware pe macOS 9
2.2 Acțiunile efectuate de malware și echivalentul lor în Endpoint Security . . 9
2.3 Comparație cu framework-uri existente de securitate 10
2.4 Alte exploit-uri ce pot fi identificate cu ES 11
2.5 Software existent ce folosește Endpoint Security 12
2.5.1 Santa 12
2.5.2 Crescendo 13
2.5.3 Shield 14
2.5.4 ProcessMonitor și FileMonitor 15

Capitolul 3 Prezentarea proiectului **16**
3.1 Tehnologii folosite 16
3.1.1 Endpoint Security 16
3.1.2 Swift 18
3.1.3 Parallels 19
3.1.4 XPC 19
3.2 Pregătirea mediului de dezvoltare 20
3.2.1 Dezactivare SIP 20
3.2.2 Dezactivare AMFI 20
3.2.3 Scrierea unui script de instalare 20
3.2.4 Atașarea unui debugger 21

3.3 Structura aplicației 23
3.3.1 XPC 24
3.3.2 Instalarea și dezinstalarea extensiei 27
3.3.3 Inițializarea clientului ES 28
3.3.4 Evenimente importante 29
3.3.5 Fluxul de detectare a unui eveniment 29
3.4 Euristici 30
3.4.1 Structură 30
3.4.2 Încărcarea euristicilor 35
3.4.3 Adăugarea de evenimente, comparatori și câmpuri 35
3.4.4 Reguli Sigma 39
3.5 Fișierul de configurare 41
3.6 Interfața grafică 42
3.6.1 Integratorul 42
3.6.2 Aplicația de testare 47

Capitolul 4 Rezultate teoretice și experimentale **50**
4.1 Suita de teste Atomic Red Team 50

Capitolul 5 Concluzii **54**

Bibliografie **55**

Capitolul 1

Introducere

1.1 Relevanța subiectului lucrării

Securitatea cibernetică este un domeniu care evoluează constant, iar pe măsură ce atacatorii descoperă tehnici noi de a exploata vulnerabilitățile unui sistem, apărătorii dezvoltă algoritmi și soluții îmbunătățite de securitate menite să blocheze atacuri existente, dar și să le prevină pe cele necunoscute de public.

În aceeași măsură, dezvoltatorii sistemelor de operare (i.e. OS) vin constant cu mitigări la nivel de sistem care limitează suprafața de atac. Dar, din cauza timpului îndelungat ce a trecut de la apariția OS-urilor folosite în ziua de astăzi, și diferența de răspândire a atacurilor de-a lungul anilor, OS-urile nu au fost concepute cu securitatea pe primul plan. De aceea, în multe cazuri este foarte greu, dacă nu imposibil, să se modifice o componentă vitală fără să afecteze compatibilitatea cu aplicațiile deja existente, lucru important având în vedere numărul de soluții software neîntreținute de o perioadă lungă de timp care încă sunt folosite în domenii importante.

Astfel, se încearcă atingerea unui echilibru între protecțiile existente în OS și cele pe care le oferă sistemele de securitate. Acestea din urmă folosesc interfețe (i.e. API-uri) speciale expuse de OS pentru a monitoriza activitatea din sistem și a impune o serie nouă de tehnici. Însă, aceste API-uri presupun ca software extern să ruleze într-un strat al OS-ului foarte apropiat de nucleu (i.e. kernel) și, deși aceste programe software pot fi considerate de încredere, nu sunt lipsite de vulnerabilități. Aici se formează un cerc vicios, în care se încearcă oferirea soluțiilor antivirus unui control cât mai apropiat asupra OS-ului, dar o vulnerabilitate în acestea ar presupune o suprafață de atac mult mai răspândită comparativ cu un atac standard.

Schimbarea acestei interfețe tradiționale de comunicare cu OS-ul presupune invalidarea componentelor kernel existente (i.e. drivere). Deși a fost o decizie controversată, Apple a adoptat această idee și a scos din uz framework-ul vechi de dezvoltat drivere de kernel (i.e. Kexts) în favoarea unor noi seturi de interfețe (i.e. DriverKit) care rulează exclusiv în modul user (i.e. user-mode). Aceasta tranziție a durat câțiva ani, timp în care dezvoltatorii au putut să migreze pe noul framework, care a adus numeroase beneficii

OS-ului. În primul rând, rulând exclusiv în user-mode, limitează exponențial suprafața de atac în cazul apariției unei vulnerabilități. Deoarece rulează similar unei aplicații standard, noul driver de user-mode (Dext) poate fi scris în orice limbaj de programare care poate interacționa cu API-uri de C (e.g. Swift, Rust), contrar vechii abordări, unde doar limbajele C și C++ puteau fi folosite, iar pentru debug se poate utiliza un debugger standard (e.g. LLDB) în locul unuia specializat pentru kernel. Atunci când există un crash, driver-ul de user-mode este restartat silențios în fundal, contrar unuia de kernel-mode, în care un crash în driver este echivalent unui crash al întregului OS.

Pe lângă DriverKit, Apple a mai lansat câteva framework-uri pentru drivere în user-mode, printre care se numără Endpoint Security (i.e. ES), specializat pe dezvoltarea soluțiilor de securitate. Construit pe un model server-client, OS-ul acționează ca un producător de evenimente pe care le trimite consumatorilor înscriși în framework-ul ES. Clienții se pot abona la două tipuri de evenimente:

- **Notificare** - Clientul este înștiințat de eveniment, fără să poată acționa asupra acestuia, iar acțiunea se desfășoară în continuare; acest tip de callback este util pentru a aduna date despre comportamentul unui proces.
- **Cerere de autorizare** - Evenimentul este pus într-o stare de pauză, iar toți clienții abonați primesc un callback prin care trebuie să decidă dacă acțiunea poate continua sau nu; acest tip de callback se folosește pentru a bloca un comportament suspect al unui proces înainte să ajungă să execute operații rău intenționate.

ES pune la dispoziție o listă bogată de evenimente, iar în fiecare an lista crește în urma actualizărilor oferite pentru OS. Majoritatea apelurilor de sistem (i.e. syscall-uri) corespund unui eveniment ES, dar există și operații mai largi care sunt integrate în acest sistem (e.g. autentificarea unui utilizator în sistem, folosirea cu succes a programului `sudo` pentru a crește privilegiile unui utilizator).

1.2 Motivație

Din cauza faptului ca dispozitivele Mac nu sunt așa populare precum cele care rulează Windows, multe soluții de securitate sunt mult mai puțin dezvoltate pentru calculatoarele Apple. Deoarece întreținerea unui modul kernel necesită o atenție specială comparativ cu o aplicație obișnuită, dezvoltatorii preferă să se axeze pe OS-ul mai des întâlnit pe dispozitivele utilizatorilor și să ofere doar o protecție de bază pentru celelalte. Un alt motiv este securitatea sporită a sistemelor macOS față de cele cu Windows: întrucât Apple nu menține compatibilitatea cu software vechi la fel e mult ca cei de la Microsoft (e.g. renunțarea la aplicațiile pe 32 de biți), le este mult mai ușor să renunțe la cod vechi și să vină cu îmbunătățiri care oferă o securitate sporită a sistemului fără să fie necesară o soluție de securitate.

Folosind noul Apple Endpoint Security Framework, multe impedimente tehnice sunt rezolvate automat, doar prin utilizarea framework-ului. Astfel, dezvoltatorii se pot axa pe

dezvoltarea de funcționalități într-un limbaj sigur din punct de vedere al memoriei (i.e. memory-safe), fără a-și face griji pentru elevarea accidentală a unei aplicații malware la nivel de kernel. Mult cod poate fi partajat între soluția de macOS și cea de Windows. Spre exemplu, tehnicile pot avea caracteristici comune, deoarece procesele au comportament similar între OS-uri. Singura diferență o reprezintă modalitatea de primire a informațiilor de la sistem (i.e. senzorii), care în contextul curent este reprezentată de un client ES.

1.3 Sisteme de detecție al comportamentului rău intenționat

1.3.1 Sistem de detecție comportamentală

Datorită multitudinii evenimentelor oferite de ES, și posibilitatea opririi execuției anumitor acțiuni, se poate dezvolta un sistem de detecție comportamentală. Pe măsură ce soluția de securitate (EDR) primește evenimente de la senzori (reprezentat de subsistemul ES), aceasta sortează și procesează evenimentele fiecărui proces, peste care aplică o serie de euristici, care asociază unei înșiruiri specifice de evenimente care sugerează o acțiune rău intenționată un anumit număr de puncte, în funcție de severitate. Odată ce un număr maxim, stabilit de EDR, este atins de un proces, acesta este oprit, utilizatorul este notificat, iar potențialele acțiuni executate asupra sistemului sunt inversate.

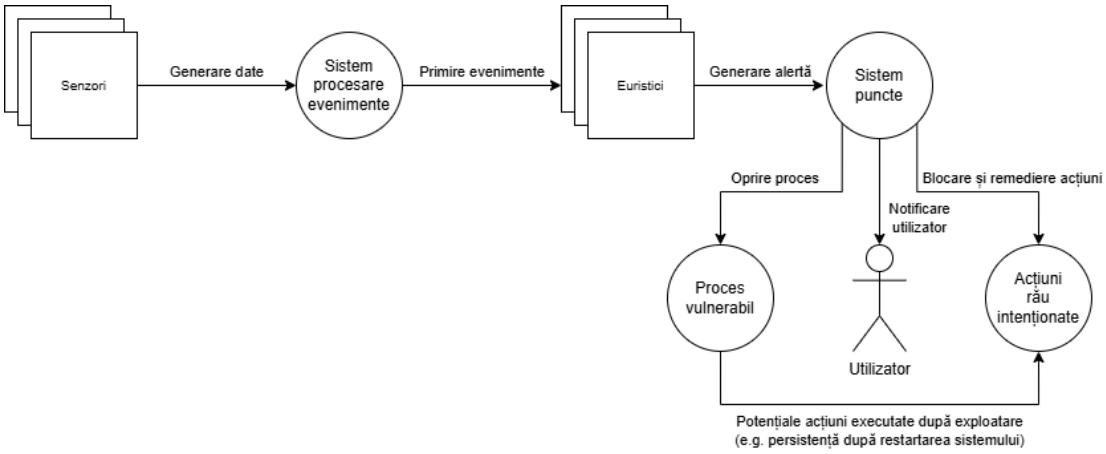


Figura 1.1: Sistem de detecție comportamentală

În figura 1.1 se află o prezentare generală a unui astfel de sistem. Un exemplu de scenariu din viața reală unde un astfel de sistem poate fi util este un virus de tip ransomware, care se poate desfășura în următorul mod:

1. Utilizatorul descarcă un binar de pe un site de phishing și îl lansează (`ES_EVENT_TYPE_NOTIFY_EXEC`).
2. Binarul descarcă următorul stadiu de pe site-ul atacatorului și îl salvează pe calculatorul victimei (`ES_EVENT_TYPE_NOTIFY_CREATE`).

3. Stadiul 2 este rulat (`ES_EVENT_TYPE_NOTIFY_FORK`, `ES_EVENT_TYPE_NOTIFY_EXEC`).

4. Deoarece pe macOS directoarele sensibile (e.g. Desktop, Downloads) sunt protejate astfel încât nici utilizatorul `root` nu le poate accesa fără permisiunea utilizatorului, virusul caută pe sistem aplicații vulnerabile la injectarea unei biblioteci dinamice (care conțin entitlement-urile `com.apple.security.cs.disable-library-validation` și `com.apple.security.cs.allow-dyld-environment-variables`). Această căutare poate genera evenimentul `ES_EVENT_TYPE_NOTIFY_READDIR`, împreună cu executarea utilitarului preinstalat `codesign` pentru a investiga entitlement-urile fiecărei aplicații.

5. Virusul descoperă o aplicație vulnerabilă căreia utilizatorul i-a acordat permisiuni de acces a directoarelor sensibile și o lansează, injectând o bibliotecă rău intenționată.

6. Ransomware-ul începe să cripteze fișierele victimei, generând o multitudine de evenimente `ES_EVENT_TYPE_NOTIFY_OPEN`, `ES_EVENT_TYPE_NOTIFY_WRITE` și `ES_EVENT_TYPE_NOTIFY_RENAME`.

Fiecare dintre acești pași poate reprezenta o euristică aplicată de soluția de securitate. Spre exemplu, descărcarea unui binar, salvarea lui pe disk și executarea lui poate reprezenta o potențială rulare a unui stadiu următor; citirea directorului `/Applications` și folosirea excesivă a binarului `codesign` poate sugera investigarea sistemului în căutarea unei aplicații vulnerabile; scrierea și redenumirea multor fișiere într-un timp foarte scurt poate descrie un comportament specific unui ransomware. Fiecare acțiune suspectă poate genera puncte asociate procesului, iar atacul poate fi oprit chiar și înainte de a putea cripta vreun fișier.

1.3.2 Sistem de detecție bazat pe injecție în procese

Deși, în majoritatea cazurilor, evenimentele produse de subsistemul ES sunt de ajuns pentru a putea scrie euristici bogate și a prinde majoritatea tipurilor de atacuri, uneori este nevoie de un control mai precis asupra funcțiilor de sistem pe care o aplicație le apelează. O multitudine de aplicații cunoscute și folosite de consumatori în mod regulat sunt scrise (sau au componente) în limbaje care nu sunt memory-safe (i.e. C, C++, Objective-C), din care pot apărea o suită de exploit-uri binare (e.g. buffer overflow, use-after-free), iar un atacator poate prelua controlul aplicației de la distanță chiar și fără interacțiunea utilizatorului. Din cauza bazei de cod vechi, aceste vulnerabilități pot exista în multe versiuni până să fie reparate.

Comportamentul unei aplicații post-exploatare poate trece nedetectat peste un sistem de detecție comportamentală, deoarece se poate manifesta într-un mod similar aplicației originale (e.g. o aplicație de tipul stocare în Cloud poate citi sau scrie mai multe fișiere pe disk fără să fie suspectă la un atac de tip ransomware). De aceea, este nevoie de o investigație la un nivel mai riguros al aplicației prin interceptarea apelurilor de sistem pe care le face aceasta.

O tehnică foarte des folosită pe sistemele Windows este injectarea unei biblioteci dinamice în procesele monitorizate, care va intercepta funcțiile de sistem prin rescrierea prologului acestora: înainte ca funcția să se execute, codul este redirecționat către codul EDR-ului, care va analiza contextul apelului și va decide dacă execuția continuă sau nu. Injectarea bibliotecii presupune obținerea unui `handle` către proces, care pe Windows este trivial. Însă, pe macOS, din cauza mitigărilor de securitate aplicate de Apple de-a lungul anilor, obținerea unui `task port` (echivalentul unui `handle`) este foarte dificilă, dacă nu imposibilă.

ES furnizează evenimentul `ES_EVENT_TYPE_AUTH_EXEC`, care este expediat la lansarea unui proces, înainte ca vreo instrucțiune de cod din acesta să fie executată. Până când ES nu primește aprobarea de la toți clienții ES, procesul nu apucă să ruleze. Deși acest lucru este nedocumentat, este posibilă obținerea unui task port către procesul suspendat, înainte de a răspunde la cererea de autorizare. Odată ce task port-ul este obținut, se poate interacționa cu memoria virtuală a procesului. Deoarece entrypoint-ul procesului încă nu a fost executat, se poate suprascrie adresa acestuia în header-ul Mach-O al procesului pentru a redirecționa execuția în codul EDR-ului, după care pașii sunt similari cu implementarea pentru Windows.

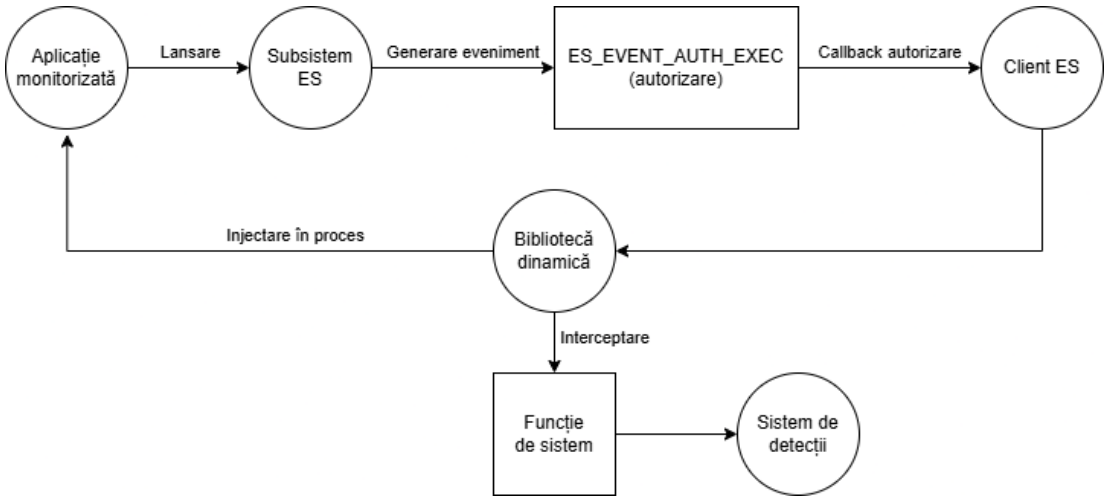


Figura 1.2: Sistem de detecție bazat pe injecție în procese

În figura 1.2 se află o diagramă a unui sistem bazat pe injecție. Un exemplu de scenariu pe care acest sistem îl poate detecta este:

1. O aplicație comercială, spre exemplu o componentă a suitei Office, prezintă o vulnerabilitate de tip buffer overflow la deschiderea unui fișier.

2. Un utilizator descarcă un fișier creat cu rea intenție care poate redirecționa execuția programului la cod specificat de atacator.

3. Aplicația este lansată, iar clientul ES injectează biblioteca dinamică în proces, care va intercepta funcțiile de sistem, precum `socket`, care este folosită deseori la stabilirea unui reverse shell.

4. Fișierul exploatează vulnerabilitatea și redirecționează execuția, incercând să apeleze funcția `socket`, însă, în schimb, ajunge în codul EDR-ului.

5. EDR-ul aplică mai multe tehnici pentru a valida apelarea funcției, iar una dintre ele observă că adresa de returnare a funcției `socket` se află pe stivă, un comportament anormal, specific unui buffer overflow.

6. Procesul este oprit, pentru a preveni următoare acțiuni rău intenționate.

Deși tehnicile din acest sistem sunt mult mai precise decât cele ale unui sistem comportamentală, implementarea acestuia este mult mai riscantă din cauza folosirii funcțiilor nedocumentate și interacționarea cu codul Assembly la un nivel foarte avansat. Astfel, trebuie ajuns la un compromis: utilizarea unui sistem de detecție comportamentala (ușor de implementat, stabil, dar cu mai puțin control asupra procesului) sau a unuia bazat pe injecție în procese (riscant, dar cu acces la o suită nouă de tehnici precise).

Însă, în fiecare an, actualizările oferite de Apple sistemului ES îmbunătățesc semnificativ funcționalitățile și tehnicile ce pot fi implementate folosind doar evenimentele primite, reducând necesitatea injectării în procese.

1.4 Obiective

Această teză propune un sistem de detecție comportamentală, capabil să detecteze atacuri moderne. Obiectivele principale atinse sunt:

- Investigarea atacurilor recente și notarea metodelor folosite de atacatori
- Implementarea unui client ES capabil să primească evenimente relevante din sistem și filtrarea acestora în funcție de proces
- Scrierea unor tehnici adecvate care pot detecta și bloca atacurile înainte să aibă vreun impact major
- Testarea întregii soluții și evaluarea eficienței asupra atacurilor moderne

1.5 Contribuții

EDR-ul prezentat, deși are unele funcționalități de bază similare cu alte produse concurente (folosind același set de API-uri oferite de sistemul ES), a fost dezvoltat cu un scop diferit față de restul, pentru a bloca atacurile folosind un sistem de detecție

comportamentală. În tabelul 1.1 sunt listate diferite funcționalități și existența lor în fiecare dintre produse:

	Soluția propusă	**Santa**	**Crescendo**	**Shield**	**ProcessMonitor**	**FileMonitor**
Abonare la evenimente	✓	✓	✓	✓	✓	✓
Vizualizare evenimente	✓	✓	✓	✗	✓	✓
Blocare injecție în procese	✓	✗	✗	✓	✗	✗
Extindere folosind euristici	✓	✗	✗	✗	✗	✗
Folosirea ambelor evenimente de notificare și autorizare	✓	✗	✗	✗	✗	✗
Suport pentru reguli Sigma	✓	✗	✗	✗	✗	✗
Testare cu tehnici Mitre	✓	✗	✗	✗	✗	✗

Tabela 1.1: Lista de funcționalități al fiecărui client ES

Soluția propusă prezintă avantaje semnificative față de produsele existente, reușind să blocheze atacuri descrise în tehnicile Mitre printr-un sistem flexibil de tehnici.

1.6 Structura tezei

Teza este structurată în următoarele capitole:

- **Capitolul 1** prezintă relevanța și evoluția sistemelor de securitate, diferența între

sistemul ES și cele existente pe alte sisteme de operare, obiectivele ce urmează a fi atinse și contribuțiile personale în domeniu.

- **Capitolul 2** descrie stadiul curent al programelor rău intenționate pe macOS, capacitatea de a detecta comportamentul lor folosind un client ES, și soluții de securitate similare deja existente.
- **Capitolul 3** detaliază structura aplicației, precum componentele și comunicarea dintre ele, tehnologiile folosite, pașii pentru a seta mediul de dezvoltare, structura euristicilor și interfața grafică.
- **Capitolul 4** analizează rezultatele testelor Atomic asupra soluției ce folosește întâi euristici convertite din reguli Sigma, și după se adaugă euristici scrise manual.
- **Capitolul 5** evidențiază obiectivele propuse și gradul de completare al acestora.

Capitolul 2

Stadiul actual în domeniu

2.1 Răspândirea aplicațiilor malware pe macOS

Deși programele de tip malware încă sunt mai predominante pentru sistemele Windows, autorii arată în articolul [1] că, recent, a avut loc o creștere majoră în atacurile pe macOS, inclusiv în complexitatea lor. Acest lucru este evidențiat în articolul [2], unde, după folosirea unui framework de analiză malware asupra binarelor folosite în atacuri reale, s-au descoperit 71 programe adware (folosind 8 certificate legitime), 2 keyloggere și 1 trojan. De asemenea, descrie metodele implementate de malware pentru a evita detecția de către EDR, precum folosirea flag-ului `PT_DENY_ATTACH` pentru funcția `ptrace` pentru a interzice atașarea unui debugger, sau detectarea rulării într-un mediu virtual (Virtual Machine - VM) prin interogarea statutului System Integrity Protection (SIP) pe care mulți analizatori de malware îl dezactivează, adunarea de informații despre hardware pentru a le potrivi cu lista de furnizori de VM-uri, sau măsurarea timpului de execuție între instrucțiuni.

2.2 Acțiunile efectuate de malware și echivalentul lor în Endpoint Security

Articolul [2] arată și câteva acțiuni comune pe care malware-ul le execută:

- Persistență
- Scriere de fișiere
- Schimbarea configurației browser-elor
- Efectuarea privilege escalation către root
- Executarea de comenzi `bash`

- Încercarea evitării analizei și a detecției de către EDR

Articolul [3] prezintă în detaliu un malware des întâlnit în mediul real și acțiunile pe care acesta le execută:

- Rularea unui script bash care decodează cu binarul `openssl` comenzi criptate cu cifrul AES și encodate folosind base64
- Descărcarea unui stadiu 2 de pe site-ul atacatorului folosind binarul `curl` și dezarhivarea acestuia folosind `unzip`

O altă implementare a malware-ului Shlayer este descrisă în articolul [4], care precizează că unul din 10 utilizatori macOS ajunge să interacționeze cu acest malware, și prezintă o implementare mai avansată, în care sunt mai multe straturi executate până ajunge ca sistemul să fie infectat complet:

- Rularea unor scripturi obfuscate de Python, care decriptează string-uri primite ca argument
- Colectarea informațiilor despre sistemul pe care rulează, precum UUID-ul hardware-ului
- Descărcarea unui stadiu 2 și folosirea comenzii `chmod` pentru a-l face executabil
- Rularea stadiului folosind comanda `open`
- Descărcarea unui alt malware de tip adware numit Cimpli, care instalează o extensie în browser-ul Safari, care injectează un script JavaScript în fiecare pagină pentru a redirecționa utilizatorii spre un domeniu rău intenționat
- Descărcarea utilitarului `mitmdump`, care este packed folosind `PyInstaller`, pentru a putea decripta tot traficul HTTPS de pe sistem prin instalarea unui Root Certificate

Toate aceste acțiuni există ca evenimente în sistemul ES, și pot fi investigate și autorizate chiar și înainte să se întâmple. Se poate observa că exploit-urile binare sunt din ce în ce mai puțin întâlnite, întrucât dezvoltatorii de aplicații tranziționează spre folosirea limbajelor memory-safe, și malware-ul este înlocuit de aplicații pe care utilizatorul este convins să le descarce.

2.3 Comparație cu framework-uri existente de securitate

Csaba Fitzl prezintă în articolul [5] diferențele dintre ES și un framework vechi de implementare a hook-urilor la nivel de apeluri de sistem numit Mandatory Access Control Framework (MACF), cu originea in TrustedBSD. Deși folosit de Apple în anumite componente de sistem, precum AppleMobileFileIntegrity (AMFI), nu este oficial susținut în

aplicații externe. Chiar și așa, este un framework foarte puternic, care permite interceptarea oricărui apel de sistem, dar care rulează exclusiv in kernel, mai exact în unul din cele mai critice puncte ale acestuia.

Endpoint Security este tot un framework care se bazează pe politici MAC, însă care abstractizează complexitatea impusă de kernel într-un framework de sine stătător ce rulează în user-mode. Folosirea ES necesită obținerea unui entitlement special de la Apple, numit `com.apple.developer.endpoint-security.client`, ce permite rularea unui client pe sistem fără a dezactiva System Integrity Protection (SIP). Acest entitlement este separat pentru dezvoltare și pentru producție și este dat doar de Apple prin completarea unui formular.

Un alt framework folosit pentru autorizarea evenimentelor de sistem este `kauth`, dar care a fost scos din uz. Deși oferă funcționalități similare ES, la fel ca MACF, rulează exclusiv în kernel, și are anumite restricții care nu sunt prezente în ES. Spre exemplu, valoarea de returnare pentru acțiunile legate de fișiere este ignorată, ceea ce face oprirea unui atac mult mai complicată [6].

2.4 Alte exploit-uri ce pot fi identificate cu ES

Datorită multitudinii de evenimente oferite de ES, se poate identifica o gamă variată de exploit-uri. Tot în articolul [5] sunt prezentate diferite exploit-uri comune și evenimentele relevante prezente in ES care ne pot ajuta să blocăm atacul:

- Injectarea unei bibliotecii dinamice rău intenționate în procese, fie prin variabile de mediu (care necesită prezența unor entitlement-uri vulnerabile în aplicația victimă), sau prin abuzarea ordinii de încărcare a biblotecilor
  - `ES_EVENT_TYPE_AUTH_EXEC` - trimis când un proces este lansat și oferă acces la variabilele de mediu furnizate procesului
  - `ES_EVENT_TYPE_AUTH_MMAP` - trimis când un fișier este încărcat în memorie (i.e. librăria dinamică)
- Obținerea unui task port al unei aplicații compilate cu anumite entitlement-uri periculoase
  - `ES_EVENT_TYPE_AUTH_GET_TASK` - trimis când un proces încearcă să obțină un task port al unui alt proces
- Rularea codului arbitrar de JavaScript în aplicațiile Electron
  - `ES_EVENT_TYPE_AUTH_EXEC` - pentru inspectarea variabilelor de mediu și a argumentelor procesului

- Folosirea symbolic links pentru a exploata verificările insuficiente ale unor aplicații privilegiate și a putea efectua scriere/ștergere arbitrară de fișiere sau creșterea privilegiilor
  - `ES_EVENT_TYPE_AUTH_LINK` - trimis când un symbolic link este creat

2.5 Software existent ce folosește Endpoint Security

2.5.1 Santa

Santa [7] este o aplicație dezvoltată de Google, capabilă să blocheze lansarea aplicațiilor suspecte in funcție de reguli setate de utilizator. Începând cu 2025, aceasta nu mai este întreținută oficial de Google, însă există fork-uri precum [8].

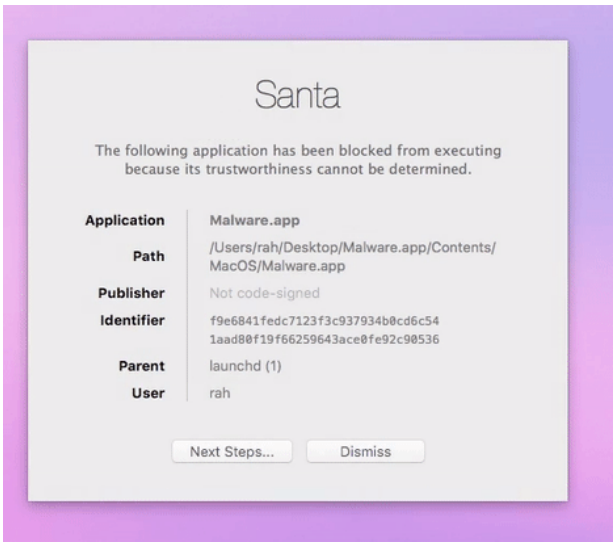


Figura 2.1: Captură de ecran din Santa la momentul lansării unei aplicații blocate [7]

În figura 2.1 se observă o alertă lansată de Santa la momentul deschiderii unei aplicații identificate ca fiind de neîncredere. Soluția poate rula în două moduri [9]:

- **modul monitor** - toate aplicațiile mai puțin cele blocate de reguli pot rula
- **modul lockdown** - doar aplicațiile permise de reguli pot rula

Deși funcționalitatea principală este blocarea de procese, Santa poate bloca și accesul la fișiere sau montarea dispozitivelor USB.

Configurarea se poate face local, aplicând reguli specifice dispozitivului pe care rulează, sau se poate seta un server numit de dezvoltatori `sync server`, unde administratorul de sistem poate defini reguli și setări care sunt propagate pe toți clienții conectați.

Componenta principală se află în clientul ES, dar Santa este compus din mai multe binare [9]:

- **Santa GUI** - Interfața grafică care afișează alerte și notificări
- **santabundleservice** - Serviciul care identifică toate aplicațiile dintr-un bundle
- **santactl** - Utilitarul pentru linia de comandă cu care se poate configura Santa și vedea informații despre acesta
- **santad** - Clientul ES care primește și autorizează evenimentele
- **santametricservice** - Serviciul care adună informații despre performanța soluției, care ajută echipa de dezvoltare
- **santasyncservice** - Serviciul care sincronizează datele cu un sync server

Santa este un utilitar bogat în funcționalități, orientat în principal către mediile enterprise, care ajută echipa de securitate să controleze activitatea de pe dispozitivele angajaților. Însă, regulile trebuie scrise manual de administratorul de sistem, și nu expune o cale de a configura reguli mai avansate, care să blocheze atacuri complexe.

2.5.2 Crescendo

Crescendo [10] este un vizualizator de evenimente, capabil să adune evenimente de la ES și să le afișeze organizat, într-un tabel, în funcție de procesul vizat.

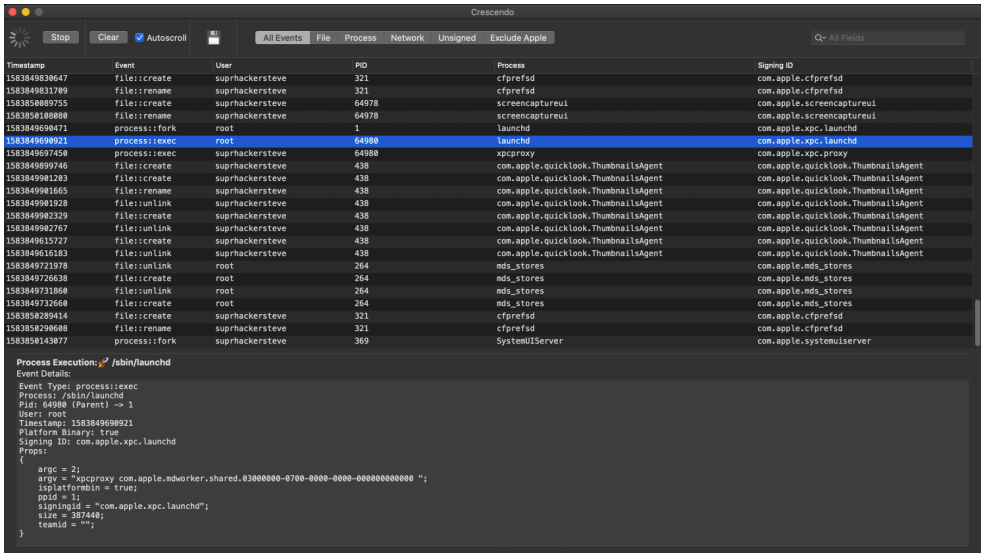


Figura 2.2: Captură de ecran din Crescendo, afișând diverse evenimente de sistem

În figura 2.2 se observă diverse evenimente generate de procesele care rulează pe dispozitiv. Evenimentele au câteva câmpuri comune (e.g. timpul, identificatorul procesului), dar dețin și informații specifice, care pot fi analizate apăsând pe fiecare eveniment în parte. Există, de asemenea, un mecanism de filtrare și căutare.

Deși nu are implementată o funcționalitate de autorizare, sau o cale de a-l configura, Crescendo este foarte util pentru a vedea și analiza evenimentele de sistem.

2.5.3 Shield

Shield [11] este un client ES care previne injectarea în procese, mai specific următoarele tehnici:

- variabile de mediu folosite pentru injectare
- obținerea unui task port prin apelarea funcției `task_for_pid`
- argumente periculoase ale aplicațiilor Electron
- abuzarea bibliotecilor dinamice
- folosirea symbolic links

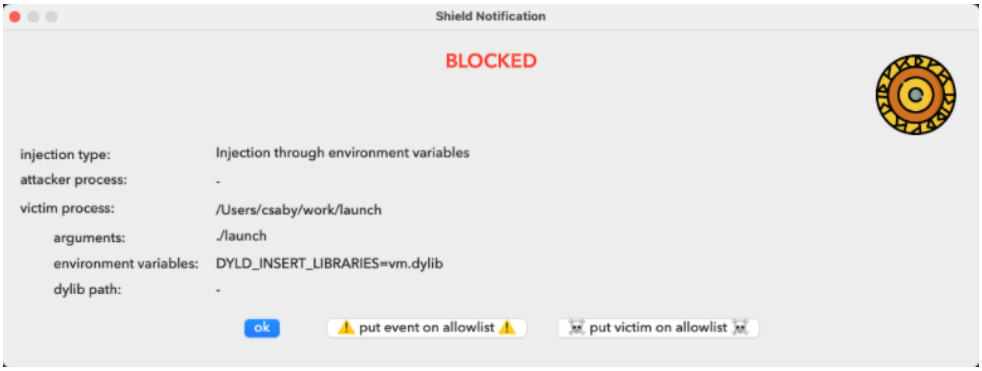


Figura 2.3: Captură de ecran din Shield, prezentând o detecție a unei variabile de mediu rău intenționate

În figura 2.3 se poate observa o alertă a aplicației Shield, blocând lansarea unui proces care folosește variabila de mediu `DYLD_INSERT_LIBRARIES`, folosită pentru a injecta o bibliotecă dinamică în proces.

Shield este o aplicație mai orientată spre exploit-uri specifice decât Santa, dar și acestuia îi lipsesc tehnici mai avansate, prezente în malware-ul real.

2.5.4 ProcessMonitor și FileMonitor

La scurt timp după ce Apple a lansat Endpoint Security, Objective-See a scris 2 articole despre scrierea unui utilitar de vizualizat evenimente legate de procese (ProcessMonitor [12]) și de fișiere (FileMonitor [13]). Articolele compară ES cu variantele vechi de a primi informații despre aceste componente:

- procese - **OpenBSM**
  - are un API destul de neintuitiv și necesită parsarea manuală a datelor binare
  - nu oferă informații complete, precum cele despre semnăturile dintr-o aplicație
  - notificările pot veni destul de târziu, iar unele procese pot deja să se fi terminat
- fișiere - **FSEvents**
  - descurajat de Apple
  - la fel ca OpenBSM, necesită parsarea de către utilizator a datelor binare
  - singura informație primită despre proces este PID-ul, care poate fi reutilizat

```
# ./processMonitor
Starting process monitor...[ok]

PROCESS EXEC ('ES_EVENT_TYPE_NOTIFY_EXEC')
pid: 7655
path: /bin/ls
uid: 501
args: (
    ls,
    "-lart",
    "."
)

signing info: {
    cdHash = 5180A360C9484D61AF2CE737EAE9EBAE5B7E2850;
    csFlags = 603996161;
    isPlatformBinary = 1 (true);
    signatureIdentifier = "com.apple.ls";
}
```

(a) ProcessMonitor

```
# ./fileMonitor
Starting file monitor...[ok]

FILE CREATE ('ES_EVENT_TYPE_NOTIFY_CREATE')

source path: (null)
destination path: /private/tmp/test

process: pid: 849
path: /usr/bin/touch
uid: 501
signing info: {
    cdHash = 818C29925EE42814EFA951413B713788AD62;
    csFlags = 603996161;
    isPlatforBinary = 1;
    signatureIdentifier = "com.apple.touch";
}
```

(b) FileMonitor

Figura 2.4: Capturi de ecran din FileMonitor, respectiv ProcessMonitor, arătând o creare de proces și fișier

Aceste aplicații au fost dezvoltate cu scop educativ, dar reprezintă resurse valoroase pentru scrierea unui client ES.

Capitolul 3

Prezentarea proiectului

3.1 Tehnologii folosite

3.1.1 Endpoint Security

Principala tehnologie folosită în proiect este Endpoint Security [14], framework-ul dezvoltat de Apple destinat soluțiilor de securitate.

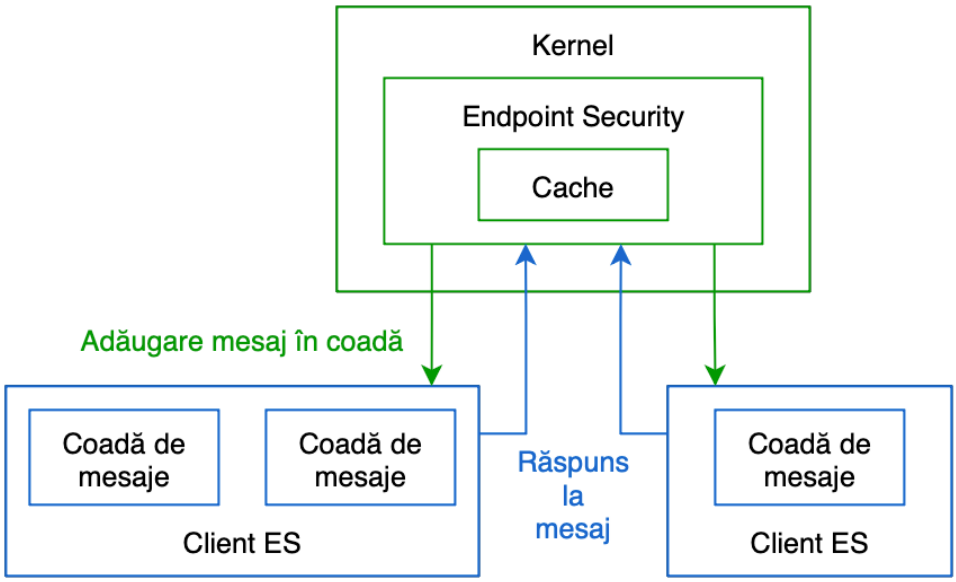


Figura 3.1: Arhitectura subsistemului Endpoint Security [15]

În figura 3.1 se află o prezentare scurtă a subsistemului ES. Acesta rulează pe principiul server-client: ES centralizează evenimentele din sistem și le propagă tuturor clienților care s-au abonat. Interfața ES este scrisă în C, pentru a permite folosirea oricărui limbaj de programare care poate interacționa cu funcții de C.

```
es_client_t *client;
es_new_client(&client, ^(es_client_t *c, const es_message_t *msg) {
    // handle event
});

es_event_type_t events[] = { ES_EVENT_TYPE_AUTH_EXEC,
↪  ES_EVENT_TYPE_AUTH_OPEN };
es_subscribe(client, events, sizeof(events) / sizeof(events[0]));
```

Listing 1: Inițializarea unui client ES și abonarea la evenimente [16]

În fragmentul de cod 1 se află codul minim necesar pentru a inițializa un client ES și a-l abona la un flux de evenimente specificate de dezvoltator. Evenimentele sunt primite prin callback-ul funcției `es_new_client`, iar prin pointer-ul la `es_client_t` se poate răspunde la un eveniment, dacă este cazul unei cereri de autorizare.

```
// răspuns simplu, de tipul permis/respins (ES_EVENT_TYPE_AUTH_EXEC)
es_auth_result_t result = should_continue(client, msg)
    ? ES_AUTH_RESULT_ALLOW
    : ES_AUTH_RESULT_DENY;
es_respond_auth_result(client, msg, ES_AUTH_RESULT_DENY, true);

// răspuns complex, bazat pe flag-uri (ES_EVENT_TYPE_AUTH_OPEN)
uint32_t authorized_flags = should_allow_write(client, msg)
    ? 0xffffffff
    : (0xffffffff & ~FWRITE);
es_respond_flags_result(client, msg, authorized_flags, true);
```

Listing 2: Răspunderea la o cerere de autorizare a unei acțiuni

Fragmentul de cod 2 descrie modalitatea de a autoriza sau a respinge un eveniment. Acest răspuns poate fi de două tipuri:

- **simplu** - evenimentele care pot fi doar permise sau respinse (e.g. lansarea unui proces)
- **complex** - evenimente care pot fi modificate granular (e.g. deschiderea unui fișier)

Fragmentul de cod folosește două funcții nedefinite (`should_continue` și `should_allow_write`), care sunt presupuse a fi parte din codul EDR-ului. Pentru un eveniment de lansare de proces, funcția decide dacă acțiunea continuă sau nu. Pentru un eveniment de deschidere a unui fișier, funcția decide dacă este permisă deschiderea acestuia în modul scriere; în acest caz, acțiunea tot va continua, însă dacă aplicația a cerut

deschiderea fișierului cu permisiuni de citire și scriere, doar cele de citire vor fi acordate. Fiecare client are un anumit termen limită pentru a răspunde la un eveniment de autorizare, iar dacă termenul este depășit, răspunsul va fi automat unul afirmativ. Subsistemul ES combină toate răspunsurile clienților ES de pe sistem care au ales să răspundă la un eveniment, și ia acțiunea cea mai restrictivă dintre toate.

Endpoint Security dispune de multe funcționalități ce nu vor fi prezentate în această lucrare, care includ (dar nu sunt limitate la):

- dezactivarea notificărilor pentru procese specifice sau pentru anumite path-uri din sistem
- utilizarea unui cache intern ES care va ține minte răspunsul precedent pentru un anumit tip de eveniment (și cu un anumit context)
- balansarea fluxului de evenimente prin mai multe subscripții sau clienți
- capacitatea de a porni odată cu sistemul de operare, inaintea restului aplicațiilor
- interoperabilitatea cu alte framework-uri bazate pe DriverKit, precum NetworkExtension, pentru a filtra inclusiv traficul de rețea

3.1.2 Swift

Swift [17] este un limbaj de programare modern apărut in 2014, scris de Apple, ajuns la a 6-a ediție. Deși a inceput ca un înlocuitor pentru Objective-C pentru a dezvolta aplicații pentru ecosistemul Apple, Swift a ajuns un limbaj pentru uz general, fiind folosit si pe alte OS-uri precum Linux și Windows, pentru servere web, dar și pentru interacțiune cu hardware-ul în cazul microcontroller-elor. Swift rulează pe Low Level Virtual Machine (LLVM) și oferă o interacțiune foarte puternică cu alte limbaje precum C, C++, Objective-C și, mai nou, Java.

Deși este constant întreținut și primește funcționalități de mai multe ori pe an, iar după 6 ediții a ajuns un limbaj potrivit pentru orice tip de scenariu, încă este folosit predominant în ecosistemul Apple, și pentru cod de logică, și pentru interfețe grafice (UI). Până la lansarea SwiftUI [18], framework-urile UIKit și AppKit erau folosite pentru a dezvolta aplicații pentru iOS și, respectiv, macOS, care, deși asemănătoare, nu puteau fiecare fi folosite pe platforme adiționale (UIKit pe macOS, și invers). SwiftUI a revoluționat scrierea interfețelor grafice printr-un framework ce poate fi utilizat pe orice platformă Apple, și a modernizat modelul interfețelor declarative prin modificări flexibile asupra limbajului Swift pentru a deveni mult mai intuitiv și ușor de învățat.

În contextul tezei, Swift va fi folosit pentru a construi majoritatea clientului ES, folosind interoperabilitatea cu limbajul C pentru a interacționa cu funcțiile expuse de framework.

3.1.3 Parallels

Parallels este cel mai cunoscut hypervisor pentru macOS, utilizat pentru a rula mașini virtuale (VM) cu macOS, Windows și Linux pe dispozitivele Apple. Oferă funcționalități bogate, care includ (dar nu sunt limitate la): gestionarea snapshot-urilor, auto-instalarea OS-urilor, partajarea fișierelor și a clipboard-ului între host și guest, interacționarea cu modul de recuperare al sistemelor macOS.

Odată cu apariția procesoarelor Apple Silicon (bazate pe arhitectura ARM) și a framework-ului Virtualization, Parallels a trecut la folosirea acestui framework pentru VM-urile cu macOS. Deoarece a fost o trecere bruscă, nu toate funcționalitățile disponibile pe procesoarele Intel sunt disponibile pentru cele Apple Silicon, însă Parallels vine deseori cu actualizări, iar capabilitățile deja existente sunt suficiente pentru a dezvolta un client ES.

Deși clienții ES rulează în user-mode și dezvoltarea acestora este posibilă pe același dispozitiv pe care se compilează aplicația fără a dezactiva System Integrity Protection (dacă dezvoltatorul deține entitlement-ul de dezvoltare pentru ES), este recomandată folosirea unei mașini virtuale. Un scenariu în care rularea clientului pe aceeași mașină ar impacta experiența de dezvoltare este uitarea accidentală de a răspunde la o solicitare de autorizare. Chiar dacă subsistemul ES va impune automat un răspuns afirmativ autorizării dacă clientul nu răspunde până la termenul limită, orice acțiune din sistem va fi încetinită până când trece acest termen, sau până când clientul ES este dezactivat. Astfel, cea mai ușoară metodă este folosirea unei mașini virtuale și a unor script-uri de instalare, și utilizarea snapshot-urilor pentru a nu fi necesară curățarea fișierelor după fiecare rulare. De asemenea, rularea într-o mașină virtuală este posibilă și fără entitlement-ul de ES, prin dezactivarea SIP și AMFI.

3.1.4 XPC

XPC [19] este o formă de comunicare între procese (IPC), specifică platformelor Apple. Deși, la bază, este un API pentru limbajul C, Apple a creat API-uri specifice Objective-C și Swift pentru a reduce din complexitatea implementării.

XPC funcționează pe modelul server-client, în care server-ul ascultă după conexiunile clienților ce pot apela funcțiile definite într-o interfață. Există două tipuri de XPC:

- **transmitere de mesaje** - Se trimit mesaje, reprezentate prin structuri care au implementat protocolul `Codable`

- **apelare de funcții** - Similar unui Remote Procedure Call (RPC), se definesc interfețe cunoscute și de client și de server; server-ul implementează funcția, iar clientul o apelează cu parametrii așteptați

3.2 Pregătirea mediului de dezvoltare

În această teză se presupune că dezvoltatorul nu deține niciun entitlement de ES, și va fi folosită o mașină virtuală pentru rularea extensiei. VM-ul trebuie să ruleze o versiune curentă de macOS, configurat astfel încât să accepte clientul ES fără entitlements valide. Asistentul de instalare automată din Parallels va seta automat un VM cu ultima versiune disponibilă de macOS.

3.2.1 Dezactivare SIP

VM-ul trebuie pornit în modul de recuperare, prin click-dreapta pe acesta (cât timp este oprit) și alegând ”Start in Recovery Mode”, similar cu figura 3.2.

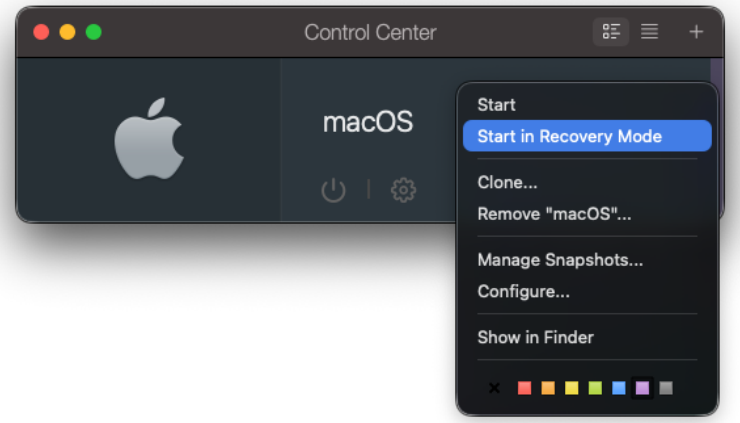


Figura 3.2: Pornirea unui VM în modul de recuperare în Parallels

Odată intrat în modul de recuperare, pentru a dezactiva SIP trebuie deschisă o linie de comandă și rulată comanda `csrutil disable`, după care sistemul poate fi restartat.

3.2.2 Dezactivare AMFI

Odată ce SIP este dezactivat, AMFI trebuie dezactivat, de asemenea, pentru a permite rularea clientului ES. Acest lucru se poate face doar din sistemul pornit în mod normal, prin deschiderea unui terminal și rularea comenzii `sudo nvram boot-args= "amfi_get_out_of_my_way=0x1"`, urmată de restartarea VM-ului.

3.2.3 Scrierea unui script de instalare

Clienții ES au anumite restricții pentru a se putea instala și rula pe sistem. Spre exemplu, aplicația care instalează clientul trebuie să fie situată în folder-ul `/Applications`.

Pentru a facilita instalarea și testarea soluției, se poate folosi un script care va copia automat aplicația compilată și o va deschide. De asemenea, se poate deschide o fereastră de linie de comandă care va rula comanda `log`, pentru a putea vedea mesajele de la clientul ES.

```
#!/bin/zsh

set -ex

# variabile de mediu
app_name=...; app_path=...; log_exec_name=...;
shared_host=...; shared_guest=...; vm=...; snapshot=...

rm -rf "$shared_host"
mkdir "$shared_host"
cp -r "$app_path" "$shared_host"

cat <<EOF > "$shared_host/$log_exec_name"
#!/bin/zsh
/usr/bin/log stream --style compact --predicate \
    'sender BEGINSWITH "Endpoint.debug" OR sender BEGINSWITH "com.toma"'
EOF

prlctl snapshot-switch "$vm" --id "$snapshot"
prlctl exec "$vm" --current-user cp -r "'$shared_guest/$app_name'"
 ↪  /Applications
prlctl exec "$vm" --current-user cp -r "'$shared_guest/$log_exec_name'"
 ↪  /tmp
prlctl exec "$vm" --current-user chmod 700 "/tmp/$log_exec_name"
prlctl exec "$vm" --current-user open "/tmp/$log_exec_name"
prlctl exec "$vm" --current-user open "/Applications/$app_name"
```

Listing 3: Script folosit pe host pentru a rula aplicația în VM

În fragmentul de cod 3 se află un script care copiază aplicația dintr-un director partajat, montat anterior, o deschide, și setează afișarea unui flux de log-uri provenite de la aplicație și extensie.

3.2.4 Atașarea unui debugger

Deoarece extensia ES rulează ca un proces standard de user-mode, este posibilă atașarea unui debugger. Din cauza faptului că aplicația este compilată pe sistemul host

și rulează într-un VM, trebuie făcuți pași adiționali pentru a stabili o sesiune de remote debugging. Singurele cerințe sunt Xcode și binarul `debugserver`, ce se află în interiorul bundle-ului Xcode. Pașii care trebuie urmați sunt:

1. [Host -> VM] Copierea binarului `debugserver` pe VM; locația acestuia este: `/Applications/Xcode.app/Contents/SharedFrameworks/LLDB.framework/Versions/A/Resources/debugserver`

2. [VM] Aflarea PID-ului extensiei (e.g. `ps aux | grep <bundle>`) și rularea `debugserver 0.0.0.0:5555 -attach=<pid>`

3. [Host] Afișarea opțiunilor avansate de debug din Xcode (`defaults write com.apple.dt.Xcode IDEDebuggerFeatureSetting 12`)

4. [Host] Modificarea unui `scheme` din Xcode pentru a selecta `Custom LLDB commands` și a rula `process connect connect://<pid>:5555` (figura 3.3)

5. [Host] Apăsarea butonului de rulare

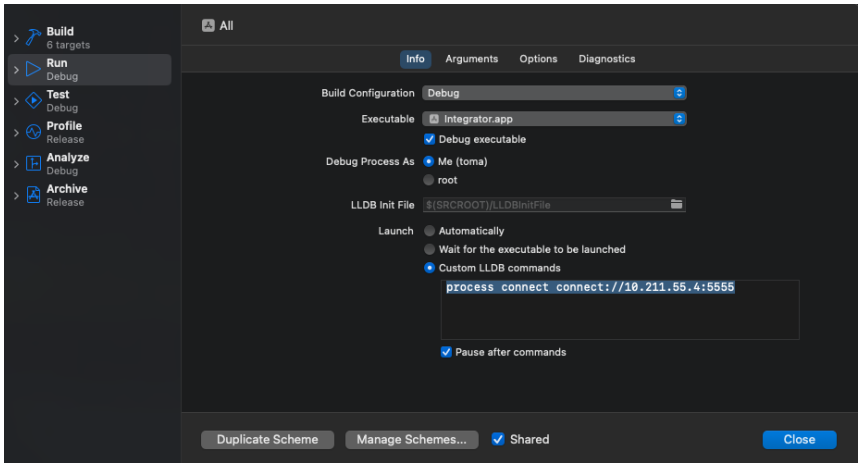


Figura 3.3: Modificarea `scheme`-ului pentru a se conecta la server-ul de debug

Acest lucru poate fi integrat în script-ul de instalare, prin scrierea unui fișier shell ce așteaptă lansarea extensiei, după care va porni `debugserver` după apăsarea unei taste de către utilizator. Este important să nu deschidem `debugserver` dacă nu intenționăm să facem o sesiune de debug, deoarece procesul va fi oprit temporar, așteptând atașarea unui debugger. Un astfel de script se poate regăsi în fragmentul de cod 4. Fișierul scris în folder-ul partajat poate fi rulat cu `prlctl exec`, similar celui ce setează fluxul de log-uri.

```
1 cat <<EOF > "$shared_host/$dbg_exec_name"
2 #!/bin/zsh
```

```
while true; do
    ps_output=\$(ps aux | grep com.toma.edr.es | grep -v grep)
    if [ \$? -eq 0 ]; then
        pid=\$(echo "\$ps_output" | awk '{ print \$2 }')
        read -s -k '?Press any key to continue.'
        sudo "$shared_guest/debugserver" 0.0.0.0:5555 --attach="\$pid"
        break
    fi
done
EOF
# ...
prlctl exec "$vm" --current-user cp -r "'$shared_guest/$dbg_exec_name'"
 ↪  /tmp
prlctl exec "$vm" --current-user chmod 700 "/tmp/$dbg_exec_name"
prlctl exec "$vm" --current-user open "/tmp/$dbg_exec_name"
```

Listing 4: Automatizarea lansării `debugserver`

3.3 Structura aplicației

Aplicația are mai multe componente, unele vizibile, altele la nivel de cod, pentru a facilita reutilizarea codului. La nivel de cod, aplicația este compusă din:

- **Integrator** - Aplicația care comunică cu extensia ES, pentru a o configura și a primi informații
- **Extensia ES** - Clientul ES care consumă evenimente de la sistem, aplică euristici, autorizează acțiuni și oprește procesele rău intenționate
- **Shared** - O librărie statică pentru a partaja date între celelalte componente; conține date precum: constante, utilități/extensii pentru funcții de sistem, definirea claselor comune etc.
- **testapp** - O aplicație pentru a testa sistemul, făcută în mod intenționat să acționeze ca o aplicație rău intenționată
- **Semnături** - O suită de fișiere care configurează sistemul, precum eurisiticile și setările aplicației

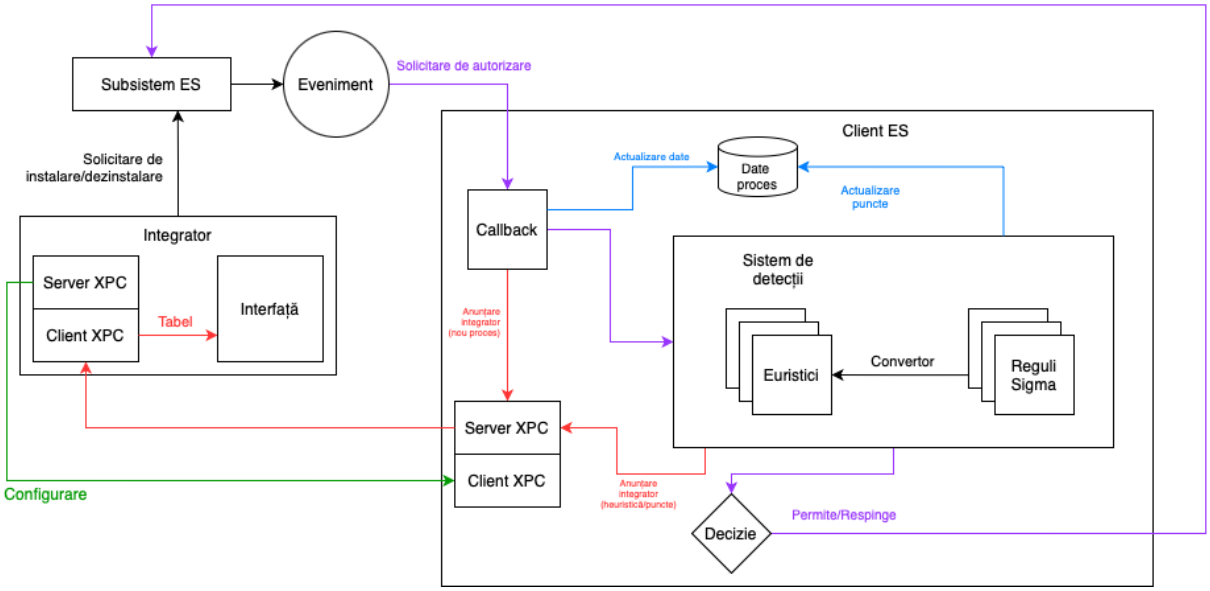


Figura 3.4: Arhitectura aplicației

3.3.1 XPC

La nivel superior, aplicația este structurată ca în figura 3.4. Cele două mari componente, integratorul și extensia, comunică între ele folosind XPC. Extensia înregistrează un server XPC folosind API-ul de Objective-C, translatat automat de Swift pentru a putea fi folosit direct. Deși, în general, comunicarea este unilaterală, iar pentru a comunica în ambele direcții ar trebui ca fiecare parte să expună un server, API-ul de Objective-C îi permite server-ului să obțină o referință la client și să ii trimită mesaje. Deși este prezent doar un singur server propriu-zis, diagrama arată fiecare componentă ca având un server și un client, pentru lizibilitate.

Extensiile ES sunt similare unui daemon `launchd`, dar diferă din anumite puncte de vedere. Unul din ele este numele serviciului XPC. O extensie ES nu poate denumi serverul XPC cu orice nume, ci trebuie să respecte convenția `<teamID>.<bundleID>`. Însă, acest lucru se poate modifica prin setarea valorii `NSEndpointSecurityMachServiceName` în fișierul `Info.plist` al extensiei, precum în figura 3.5.

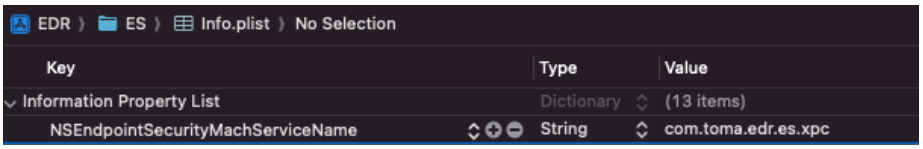


Figura 3.5: Setarea unui nume diferit al serviciului XPC

Fiecare parte din comunicarea XPC definește un protocol (o interfață), cunoscut și de server, și de client, similare cu cele din fragmentul de cod 5.

```swift
@objc(XPCIntegratorToExtension)
public protocol XPCIntegratorToExtension {
    func checkIn()
}

@objc(XPCExtensionToIntegrator)
public protocol XPCExtensionToIntegrator {
    func checkIn()
}
```

Listing 5: Cele 2 protocoale definite de integrator și de extensie

Server-ul deschide o conexiune folosind clasa `NSXPCListener` și setează ca `delegate` o clasă care implementează protocolul `NSXPCListenerDelegate`. Metoda `listener(_:shouldAcceptNewConnection:)` din protocol se apelează atunci când un client încearcă să deschidă o conexiune, și este responsabilă de a valida și de a obține informații referitoare la conexiune. Un exemplu de implementare se regăsește în fragmentul de cod 6.

```swift
// ...undeva în clasă
let listener = NSXPCListener(machServiceName: kXPCDriver)
listener.delegate = self
listener.resume()
// ...

func listener(
    _ listener: NSXPCListener,
    shouldAcceptNewConnection newConnection: NSXPCConnection
) -> Bool {
    newConnection.exportedInterface = NSXPCInterface(with:
    ↪ XPCIntegratorToExtension.self)
    newConnection.exportedObject = self
    newConnection.remoteObjectInterface = NSXPCInterface(with:
    ↪ XPCExtensionToIntegrator.self)
    newConnection.setCodeSigningRequirement("[...]")
    newConnection.resume()
    self.integratorProxy = newConnection.remoteObjectProxyWithErrorHandler
    ↪ { error in
        // ...
    } as? XPCExtensionToIntegrator
    return true
}
```

Listing 6: Server XPC

Serverul definește cele 2 protocoale, pentru a ști ce funcții se apelează și poate apela. Validarea se face prin apelarea funcției `setCodeSigningRequirement`, ce descrie informații despre client, pentru a preveni conectarea altor clienți neautorizați. Dat fiind faptul că serverele XPC, de obicei, efectuează acțiuni privilegiate, este important ca singurul client ce se poate conecta să fie cel autorizat de server. Argumentul funcției, de regulă, are formatul: `identifier "<bundle ID>" and anchor apple generic and certificate leaf[subject.CN] = "<developer certificate>"`, și validează bundle identifierul clientului și faptul că este semnat de un anumit dezvoltator. La final, serverul obține o referință la implementarea unei interfețe ale clientului prin apelarea funcției `remoteObjectProxyWithErrorHandler`, pe care o poate folosi pentru a apela funcții din client.

Odată ce serverul a deschis canalul XPC, clientul se poate conecta folosind clasa `NSXPCConnection`, precum în fragmentul de cod 7.

Listing 7 Client XPC

```
1 self.connection = NSXPCConnection(machServiceName: kXPCDriver, options:
  ↪  .privileged)
2 self.connection.remoteObjectInterface = NSXPCInterface(with:
  ↪  XPCIntegratorToExtension.self)
3 self.connection.exportedInterface = NSXPCInterface(with:
  ↪  XPCExtensionToIntegrator.self)
4 self.connection.exportedObject = self
5 self.connection.setCodeSigningRequirement("[...]")
6 self.connection.resume()
7 self.service = connection.remoteObjectProxyWithErrorHandler { error in
8     // ...
9 } as? XPCIntegratorToExtension
```

Similar pregătirii server-ului, clientul setează cele două interfețe, validează autenticitatea server-ului, și obține o referință la o implementare a interfeței acestuia, pe care poate apela funcții.

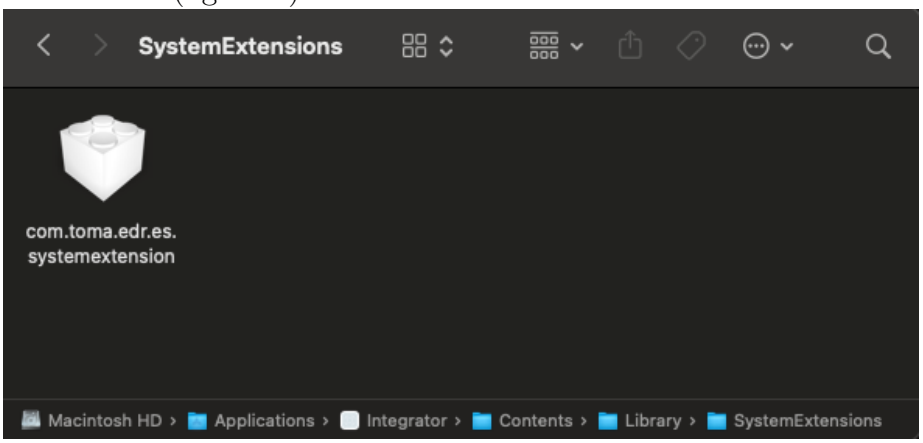
În cadrul soluției de securitate, XPC este folosit pentru următoarele scenarii:

- **Integrator -> Extensie**: Aplicarea configurărilor ”on-demand”; în cazul celor afișate în interfață utilizatorului, integratorul poate notifica extensia că o anumită setare trebuie modificată
- **Extensie -> Integrator**: Transmiterea acțiunilor importante întâmplate în sistem (e.g. pornirea/oprirea unui proces, potrivirea cu succes a unei euristici, terminarea

forțată a unui proces la atingerea numărului maxim de puncte)

3.3.2 Instalarea și dezinstalarea extensiei

Contrar altor sisteme de operare, unde driverele vin separat de celelalte componente și se pot instala folosind un script, pe macOS extensia ES vine incorporată în integrator, care este responsabil pentru instalarea și dezinstalarea extensiei. Integratorul trebuie să stea în folder-ul `/Applications`, altfel subsistemul ES va respinge orice cerere. Xcode se ocupă automat de incorporarea extensiei, și o pune în `Contents/Library/SystemExtensions/<bund ID>.systemextension` (figura 8)



Listing 8: Locația extensiei în interiorul aplicației, așa cum este afișată în Finder

Instalarea unei extensii se face prin apelarea funcției `OSSystemExtensionRequest.activationRequest`, setarea unui `delegate` care implementează protocolul `OSSystemExtensionRequestDelegate` și trimiterea solicitării folosind `OSSystemExtensionManager.shared.submitRequest`. Protocolul oferă callback-uri pentru diferite scenarii în care se poate afla instalarea extensiei:

- așteptare aprobare de la utilizator
- instalare cu succes
- apariția unei erori
- instalarea unei noi versiuni a extensiei

Un exemplu de implementare se află in fragmentul de cod 9. Dezinstalarea este similară, folosind funcția `OSSystemExtensionRequest.deactivationRequest`.

```
1 // undeva în clasă...
2 let request = OSSystemExtensionRequest.activationRequest(
3     forExtensionWithIdentifier: kExtensionBundleIdentifier,
```

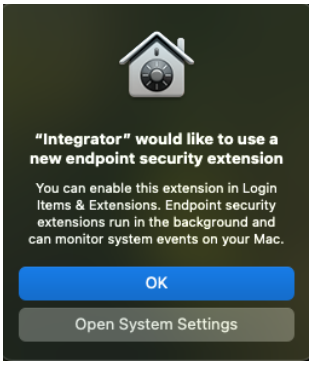
```
    queue: .main
)
request.delegate = self
OSSystemExtensionManager.shared.submitRequest(request)
// ...

extension Installer: OSSystemExtensionRequestDelegate {
    // ...
}
```

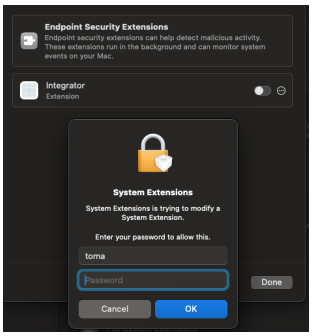
Listing 9: Instalarea unei extensii ES

După trimiterea cererii de instalare, sistemul notifică utilizatorul că o aplicație încearcă să instaleze o extensie de securitate (figura 3.6a):

Al doilea buton redirecționează utilizatorul către secțiunea relevantă din setări, unde poate permite instalarea extensiei, confirmând acțiunea utilizând parola contului (figura 3.6b)



(a) Notificarea după trimiterea cererii



(b) Aprobarea instalării extensiei

Figura 3.6: Procesul instalării unei extensii ES

Acesta este singurul mod prin care o extensie ES poate fi instalată în sistem, și doar cu permisiunea utilizatorului. Instalarea nu se poate automatiza, nici cu privilegiile user-ului `root`, pentru a preveni instalarea unei extensii rău intenționate.

3.3.3 Inițializarea clientului ES

Deși ES este un API scris pentru limbajul C, orice alt limbaj care poate interacționa cu acesta îl poate apela. În cazul Swift, translatarea API-ului este făcută automat, și se pot apela funcțiile direct din cod, iar compilatorul mapează tipurile de date din C la tipuri similare din Swift (exemplu în fragmentul de cod 10).

```c
// C
es_new_client_result_t
es_new_client(
    es_client_t *_Nullable *_Nonnull client,
    es_handler_block_t _Nonnull handler
);
```

```swift
// Swift
public func es_new_client(
    _ client: UnsafeMutablePointer<OpaquePointer?>,
    _ handler: @escaping es_handler_block_t
) -> es_new_client_result_t
```

Listing 10: Translatarea unei funcții de C în Swift

Configurarea sistemului ES se face în aceeași modalitate ca în C: se apelează `es_new_client`, primind un pointer către client, care poate fi folosit pentru a apela `es_subscribe` cu evenimentele de interes pentru soluția de securitate.

3.3.4 Evenimente importante

Pe lângă evenimentele specifice de interes pentru fiecare EDR, două evenimente sunt aproape mereu vitale: crearea și terminarea unui proces. Deoarece un proces (numit uneori `task` pe macOS) reprezintă cea mai mică unitate pe care un OS o poate gestiona (exceptând thread-urile, care rulează sub un proces), este important ca o soluție de securitate să țină cont de fiecare proces care se execută.

În contextul soluției de securitate prezentate, EDR-ul interceptează evenimentele `ES_EVENT_TYPE_NOTIFY_EXEC` și `ES_EVENT_TYPE_NOTIFY_EXIT`. Atunci când un proces se lansează, extensia creează o structură în care menține informații despre proces (e.g. argumente, PID, calea executabilului). La momentul ieșirii unui proces, extensia obține exit code-ul. La fiecare modificare în lista de informații despre procese, extensia notifică integratorul, care le afișează într-un tabel în funcție de relația parent-child.

3.3.5 Fluxul de detectare a unui eveniment

Tot în figura 3.4 este descris mecanismul prin care EDR-ul analizează evenimentele primite de la subsistemul ES și decide dacă un proces execută o acțiune legitimă. În interiorul bundle-ului cu soluția de securitate se află o serie de euristici destinate să detecteze un anumit tip de comportament, ce vor fi explicate în detaliu într-o secțiune ulterioară. Fiecare euristică are asociat un anumit număr de puncte, ce va fi adunat procesului în cazul potrivirii acesteia. La orice euristică care s-a potrivit, integratorul este notificat,

pentru a afișa în interfață toate euristicile asociate unui proces, precum și numărul curent de puncte.

La atingerea unui număr prestabilit de puncte, procesul este considerat a fi rău intenționat și este oprit. EDR-ul încearcă să oprească procesul prin apelarea funcției `kill` cu semnalul `SIGKILL` (valoarea 9), care va opri procesul fără a apela niciun callback din acesta, contrar altor semnale. La oprirea unui proces, integratorul este notificat, pentru a reflecta schimbările în interfață.

În funcție de setările EDR-ului, extensia se poate abona la evenimentul de autorizare de lansare a unui proces în loc de cel de notificare. Echivalentul pentru evenimentul de ieșire nu există, întrucât oprirea unui proces nu poate fi întreruptă. Avantajul abonării la evenimentele de autorizare îl constituie faptul că, în cazul în care un proces este marcat ca rău intenționat, acesta poate fi prevenit în a face alte acțiuni (în acest context, să creeze un proces nou). Deoarece în cazul evenimentelor de notificare acestea sunt trimise asincron, poate exista un race condition în momentul în care soluția de securitate oprește procesul și alte acțiuni pe care procesul încearcă să le execute, și, astfel, să ruleze acțiuni pentru o perioadă scurtă de timp, deși a fost identificat ca fiind rău intenționat. În schimb, evenimentele de autorizare sunt expediate sincron și așteaptă răspunsul tuturor clienților ES, iar clientul poate refuza orice eveniment provenit de la aplicația în cauză, până reușește să o oprească.

3.4 Euristici

3.4.1 Structură

EDR-ul este îndeajuns de flexibil să acomodeze o gamă variată de euristici. Acestea sunt scrise în limbajul YAML, și conțin câteva câmpuri obligatorii:

- **name** - numele euristicii; convenția este să fie numită folosind `screaming snake case` și să înceapă cu `HEUR_`
- **points** - numărul de puncte adunate procesului în cazul în care euristica se potrivește
- **tags** - etichete asociate euristicii, spre exemplu tehnici MITRE; sunt similare cu comentariile
- **onDuplicate** - ce se întâmplă dacă euristica se aplică de mai multe ori unui proces; valorile pot fi:
  - **ignore** - nici punctele, nici euristica nu sunt aplicate a doua oară
  - **ignorePoints** - euristica este adăugată în lista procesului, dar nu se adaugă iar punctele
  - **add** - și euristica, și punctele sunt adăugate

- **match** - ce acțiuni sunt asociate euristicii

O euristică poate asocia două tipuri de acțiuni: un eveniment sau alte euristici.

Euristici bazate pe evenimente

O euristică poate analiza un eveniment și poate aplica anumite restricții pe argumentele evenimentului. Un obiect din lista de `match` care descrie un eveniment are următoarele câmpuri:

- **name** - numele evenimentului; în general, este identic cu numele evenimentului trimis de ES, fără prefixul `ES_EVENT_TYPE_NOTIFY_` (e.g. `READDIR`, `EXEC`)
- **restrict** - lista de restricții aplicată evenimentului; acest dicționar trebuie să aibă o singură cheie, dar poate fi definit recursiv, folosind următorii termeni:
  - **anyOf**: oricare restricție din lista ce urmează se poate potrivi evenimentului
  - **allOf**: toate restricțiile din listă trebuie să fie aplicate
- **threshold** - de câte ori trebuie să se aplice evenimentul pentru ca euristica să fie considerată; momentan, poate avea doar valoarea `total`, cu următoarele câmpuri, dar poate fi cu ușurință adaptat la alte tipuri
  - **amount** - câte evenimente trebuie sâ răspundă euristicii
  - **interval** (opțional) - în câte secunde trebuie să apară acele evenimente; dacă câmpul nu există, se ia lista completă de evenimente

O restricție este formată din cheie și valoare, cu următoarea structură:

- **cheie** - `<câmp>+<comparator>`
  - **câmp** - un anumit atribut al evenimentului; de exemplu, pentru un eveniment de tip `EXEC`, câmpul `path` descrie calea către fișierul executat
  - **comparator** - modul prin care se compară valoarea definită cu valoarea din eveniment; de exemplu, `equals` compară cele două șiruri de caractere în întregime, `contains` compară dacă un șir de caractere sau o listă conține o anumită valoare
- **valoare** - șirul de caractere care va fi comparat cu atributul din eveniment

O descriere alternativă a formatului euristicilor se găsește în fragmentul de cod 11, ce prezintă structura în formatul BNF (Backus-Naur Form).

```
rule              ::= "name:" string
                      "points:" integer
                      "onDuplicate:" duplicate_action
                      "tags:" tag_list
                      "match:" match_list
duplicate_action  ::= "ignore" | "ignorePoints" | "add"
tag_list          ::= string { string }
match_list        ::= match_entry { match_entry }
match_entry       ::= "event:" event
                    | "heuristic:" heuristic
event             ::= "name:" string
                      "threshold:" threshold
                      "restrict:" restrict_item
heuristic         ::= "name:" string
                      "threshold:" threshold
threshold         ::= "total:" threshold_total
threshold_total   ::= "amount:" integer
                      [ "interval:" integer ]
restrict_item     ::= "allOf:" restrict_list
                    | "anyOf:" restrict_list
                    | key_value
restrict_list     ::= restrict_item { restrict_item }
key_value         ::= key ":" string
key               ::= field "+" comparator
field             ::= identifier
comparator        ::= identifier
identifier        ::= letter { letter | digit | "_" }
string            ::= { character }
character         ::= any printable character except newline
integer           ::= digit { digit }
digit             ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8"
  ↪  | "9"
letter            ::= "a" | ... | "z" | "A" | ... | "Z"
```

Listing 11: Formatul BNF al euristicilor

EDR-ul mapează valorile evenimentelor și a câmpurilor din restricții la valorile reale trimise de subsistemul ES. Deoarece, la nivel de cod, comparatorii și atributele evenimentului sunt definite separat, este foarte ușoară adăugarea unor noi valori care pot fi folosite în orice euristică, prin simpla completare a unei noi intrări într-un dicționar.

Un exemplu de euristică bazată pe evenimente este descris în fragmentul de cod 12. Aceasta definește euristica `HEUR_INSPECT_ENTITLEMENTS`, îi atribuie 20 de puncte, iar în

cazul în care se aplică de mai multe ori, nici ea, nici punctele nu sunt aplicate încă o dată. Se uită după evenimentul `EXEC`, doar dacă calea executabilului este `/usr/bin/codesign`, iar unul din argumente este `--entitlements`. Pentru a fi potrivită, trebuie să fie cel puțin 5 evenimente în ultima secundă. Această euristică identifică acțiunea unui malware de a inspecta aplicațiile deja existente pentru a găsi un entitlement vulnerabil.

```
name: HEUR_INSPECT_ENTITLEMENTS
points: 20
onDuplicate: ignore
tags: []
match:
- event:
    name: EXEC
    restrict:
      allOf:
        - path+equals: "/usr/bin/codesign"
        - args+contains: "--entitlements"
    threshold:
      total:
        interval: 1
        amount: 5
```

Listing 12: Euristică bazată pe un eveniment

Euristici bazate pe alte euristici

O euristică poate analiza, de asemenea, prezența altor euristici. De exemplu, o euristică poate descrie un comportament malițios doar în anumite scenarii, sau doar dacă mai este prezentă încă o euristică. Un obiect care descrie prezența unei alte euristici are următoarele câmpuri:

- **name** - numele euristicii
- **threshold** - identic cu euristicile bazate pe evenimente

Fragmentul de cod 13 descrie o astfel de euristică, numită `HEUR_AGGRESSIVE_ENTITLEMENT_SEARCH`. Aceasta atribuie 50 de puncte procesului, dacă si euristicile `HEUR_ENUM_APPS` și `HEUR_INSPECT_ENTITLEMENTS` sunt deja prezente măcar o dată.

```
name: HEUR_AGGRESSIVE_ENTITLEMENT_SEARCH
points: 50
onDuplicate: ignore
```

```
4  tags: []
5  match:
6  - heuristic:
7      name: HEUR_ENUM_APPS
8      threshold:
9        total:
10         amount: 1
11 - heuristic:
12     name: HEUR_INSPECT_ENTITLEMENTS
13     threshold:
14       total:
15         amount: 1
```

Listing 13: Euristică bazată pe un eveniment

Euristica `HEUR_ENUM_APPS` este definită în fragmentul de cod 14 și descrie acțiunea unei aplicații de a enumera directorul `/Applications`, ce conține toate aplicațiile instalate de utilizator.

```
1  name: HEUR_ENUM_APPS
2  points: 10
3  onDuplicate: ignore
4  tags: []
5  match:
6  - event:
7      name: READDIR
8      restrict:
9        allOf:
10         - path+equals: "/Applications"
11     threshold:
12       total:
13         amount: 1
```

Listing 14: Euristică bazată pe un eveniment

Cele două euristici asociate pot fi inofensive dacă sunt prezente separat, dar împreună marchează un comportament suspect. Enumerarea de aplicații nu este, neapărat, o acțiune rău intenționată, pentru aplicațiile de tip ”file manager”. Inspectarea entitlement-urilor unei aplicații, de asemenea, nu este un comportament strict rău, deoarece o aplicație poate verifica integritatea altor aplicații asociate (e.g. un mecanism de ”auto-update”). Dar, când un binar enumeră celelalte aplicații instalate și imediat încearcă sa inspecteze entitlement-urile descrie o probabilitate foarte mare de un atac.

3.4.2 Încărcarea euristicilor

Euristicile se află în bundle-ul cu extensia, care se află în bundle-ul cu aplicația principală, așa cum este prezentat în figura 3.7.

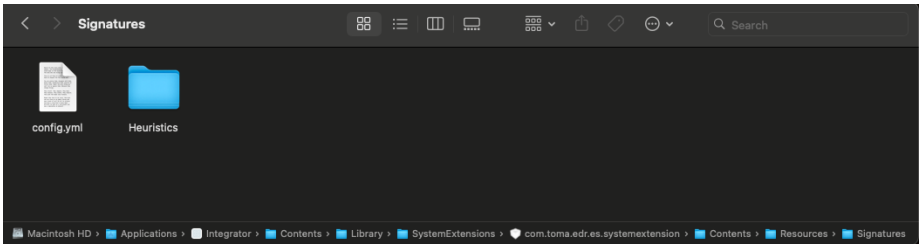


Figura 3.7: Locația euristicilor

Euristicile sunt copiate automat de Xcode la momentul compilării sistemului, si sunt încărcate de extensia ES la lansare. Deoarece macOS protejează conținutul unei aplicații împotriva modificărilor efectuate de alte aplicații fără autorizarea utilizatorului, este un loc bun pentru a le stoca fără ca un atacator să le poată schimba.

3.4.3 Adăugarea de evenimente, comparatori și câmpuri

Fiecare element din meta-limbajul euristicilor ce se referă la (sau folosește) o caracteristică a ES trebuie descris în cod, pentru a forma o asociere între șirul de caractere parsat din semnături și structurile din limbajul Swift. Din cauza caracteristicii de static typing din Swift, nu este atât de ușor a amesteca mai multe tipuri de date simultan, dar Swift ne permite să definim o ierarhie organizată de tipuri datorită funcționalităților bogate ale type system-ului acestuia.

Evenimente

Evenimentele sunt ținute într-o listă cu elemente de tipul `EventInfo`, descrisă în fragmentul de cod 15.

```
1  public enum EventType {
2      case notify
3      case notifyOnly
4      case authSimple
5      case authFlags
6  }
7
8  open class EventInfo {
9      public var heurName: String
10     public var esValue: es_event_type_t
```

```
11      public var type: EventType
12      // ...
13  }
```

Listing 15: Structura unui eveniment din semnături

Structura reține atât numele evenimentului ce va fi scris în fișierul cu euristica, cât și tipul ce va fi trimis de subsistemul ES. De asemenea, asociază fiecărui eveniment un tip, pentru a diferenția între evenimentele de autorizare și cele de notificare, cât și tipul de răspuns pe care îl așteaptă ES. Valorile pot fi:

- **notify** - eveniment de notificare
- **notifyOnly** - eveniment de notificare, fără alternativă de autorizare
- **authSimple** - eveniment de autorizare ce așteaptă un răspuns de tipul permis/respins
- **authFlags** - eveniment de autorizare ce așteaptă un răspuns de tip ”flags”

În lista `eventMapping` se țin structurile de tip `EventInfo`, care stochează atât evenimentele de autorizare, cât și cele de notificare. În funcție de cum este configurat EDR-ul, se poate abona la oricare din cele două tipuri de evenimente. Valoarea `notifyOnly` este importantă în cazul abonării la evenimentele de autorizare, iar unele evenimente sunt disponibile doar în varianta de notificare; în acest caz, EDR-ul nu are altă alternativă decât să se aboneze la evenimentul de notificare. Un exemplu de definire a acestor structuri pentru diferite evenimente este prezent în fragmentul de cod 16.

```
1  public let eventMapping: [EventInfo] = [
2      .init(heurName: "EXEC", esValue: ES_EVENT_TYPE_NOTIFY_EXEC, type:
       ↪  .notify),
3      .init(heurName: "EXEC", esValue: ES_EVENT_TYPE_AUTH_EXEC, type:
       ↪  .authSimple),
4      .init(heurName: "EXIT", esValue: ES_EVENT_TYPE_NOTIFY_EXIT, type:
       ↪  .notifyOnly),
5      .init(heurName: "OPEN", esValue: ES_EVENT_TYPE_NOTIFY_OPEN, type:
       ↪  .notify),
6      .init(heurName: "OPEN", esValue: ES_EVENT_TYPE_AUTH_OPEN, type:
       ↪  .authFlags),
7      // ...
8  ]
```

Listing 16: Diverse instanțe ale structurii `EventInfo`

Comparatori

Un comparator este, la bază, o funcție ce primește o valoare, o compară cu altă valoare, și returnează un rezultat de tip `Bool` dacă comparația a avut succes sau nu. Însă, pentru a avea un sistem de euristici cât mai flexibil, este necesară prezența mai multor tipuri de date. Spre exemplu, un proces poate avea un câmp de tip șir de caractere (e.g. numele), dar și alt câmp de tip listă de șiruri de caractere (e.g. argumentele procesului). Swift ne permite să lucrăm cu mai multe tipuri de date prin definirea unui protocol cu un `associatedtype`, precum în fragmentul de cod 17.

```
public protocol AttributeComparator {
    associatedtype Attribute
    var extractor: (es_message_t) -> Attribute { get }

    subscript(comparator: String, message: es_message_t) -> (String) ->
    ↪ Bool { get }
}
```

Listing 17: Protocolul de bază al unui comparator

Protocolul forțează clasele ce îl implementează să aibă două caracteristici: o variabilă de tip funcție capabilă să extragă o anumită valoare dintr-un mesaj ES și o funcție de tip `subscript` (similar unui dicționar), ce va lua ca argumente numele comparatorului și mesajul ES și va returna o funcție ce are ca date de intrare valoarea ce va fi comparată, și ca date de ieșire rezultatul. Un prim exemplu este clasa `StringComparator`, ce va extrage un câmp de tip `String` (fragmentul de cod 18).

```
public class StringComparator: AttributeComparator {
    public var extractor: (es_message_t) -> String

    init(extractor: @escaping (es_message_t) -> String) {
        self.extractor = extractor
    }

    public subscript(comparator: String, message: es_message_t) ->
    ↪ (String) -> Bool {
        let field = self.extractor(message)
        let handler = Self.apply(field: field, comparator: comparator)
        guard let handler else {
            SharedLog.fatal("Cannot find comparator for \(comparator) in ⌟
            ↪ \(Self.self)")
        }
```

```
14          return handler
15      }
16
17      static func apply(field: String, comparator: String) -> ((String) ->
        ↪ Bool)? {
18          return [
19              "equals": { arg in
20                  field == arg
21              },
22               "regex": { arg in
23                  let regex = try? Regex(arg)
24                  guard let regex else {
25                      SharedLog.fatal("Cannot parse regex \(arg)")
26                  }
27                  return field.firstMatch(of: regex) != nil
28              }
29              // ...
30          ][comparator]
31      }
32  }
```

Listing 18: Implementarea unui comparator de tip `String`

Clasa definește extractor-ul ca având rezultatul `String` (indeplinind cererea protocolului de a avea un `associatedtype`) și declară un dicționar ce va descrie cum va fi fiecare comparator utilizat. Spre exemplu, comparatorul `equals` face o simplă egalitate între șiruri de caractere, iar cel `regex` va aplica o expresie regulată asupra câmpului. Definirea unui comparator de tip listă de șiruri de caractere este similară, înlocuind `associatedtype`-ul cu tipul `[String]` și definind noi comparatori specifici acestui tip.

Câmpuri

Din cauza faptului că API-ul de ES provine din C, și Swift este un limbaj ce folosește static typing, nu putem accesa direct atributele unui mesaj ES după nume, iar uneori avem nevoie de câmpuri mai complexe, ce nu există direct în ES. De aceea, folosind comparatorii putem defini ce câmpuri putem referenția din euristici intr-un dicționar de dicționare: prima cheie reprezintă numele evenimentului, iar cheia valorii primului dicționar se referă la numele câmpului. O implementare pentru câteva evenimente și câmpuri se regăsește în fragmentul de cod 19.

```
1   let attrsMapping: [String: [String: any AttributeComparator]] = [
2       "EXEC": [
```

```
        "path": StringComparator { message in
            String.fromEs(message.event.exec.target.pointee.executable.po⌋
            ↪ intee.path) ?? ""
        },
        "args": StringArrayComparator { message in
            message.event.exec.args
        },
        // ...
    ],
    "OPEN": [
        "file": StringComparator { message in
            String.fromEs(message.event.open.file.pointee.path) ?? ""
        },
        // ...
    ],
    // ...
]
```

Listing 19: Definirea câmpurilor evenimentelor

În acest exemplu se definesc câteva argumente pentru două evenimente. Evenimentul `EXEC` are un câmp de tip `String` numit `path`, ce se referă la calea executabilului, și un câmp de tip `[String]`, ce returnează lista de argumente, iar evenimentul `OPEN` are un câmp de tip `String` ce reprezintă calea fișierului ce se dorește a fi deschis.

3.4.4 Reguli Sigma

Regulile Sigma [20] reprezintă un standard pentru semnături folosite de soluțiile de securitate pentru a detecta evenimente rău intenționate din sistem. Repository-ul Sigma de pe GitHub conține o colecție bogată de reguli pentru fiecare sistem de operare, și se potrivește perfect unui sistem bazat pe evenimente.

Deoarece formatul regulilor Sigma nu oferă toate informațiile necesare unei euristici (e.g. numărul de puncte), EDR-ul prezentat oferă un script de conversie a unei reguli Sigma în euristici compatibile.

```
title: System Integrity Protection (SIP) Disabled
id: 3603f18a-ec15-43a1-9af2-d196c8a7fec6
status: test
description: |
    Detects the use of csrutil to disable the Configure System Integrity
    ↪ Protection (SIP). This technique is used in post-exploit
    ↪ scenarios.
```

```
references:
    - https://ss64.com/osx/csrutil.html
    - https://objective-see.org/blog/blog_0x6D.html
    - https://www.welivesecurity.com/2017/10/20/osx-proton-supply-chain-attack-elmedia/
    - https://www.virustotal.com/gui/file/05a2adb266ec6c0ba9ed176d87d8530e71e845348c13caf9f60049760c312cd3/behavior
author: Joseliyo Sanchez, @Joseliyo_Jstnk
date: 2024-01-02
tags:
    - attack.discovery
    - attack.t1518.001
logsource:
    product: macos
    category: process_creation
detection:
    # VT Query: behavior_processes:"csrutil status" p:5+ type:mac
    selection:
        Image|endswith: '/csrutil'
        CommandLine|contains: 'disable'
    condition: selection
falsepositives:
    - Unknown
level: medium
```

Listing 20: Exemplu de regulă Sigma

În fragmentul de cod 20 se află un exemplu al unei reguli Sigma luată din repository-ul oficial. Regula identifică deschiderea procesului `csrutil` cu argumentul `disable`, ce descrie intenția de a dezactiva System Integrity Protection (SIP).

```
name: HEUR_SYSTEM_INTEGRITY_PROTECTION_(SIP)_DISABLED
points: 0
onDuplicate: ignore
tags:
- attack.discovery
- attack.t1518.001
match:
- event:
    name: EXEC
    threshold:
      total:
```

```
12          amount: 1
13      restrict:
14        allOf:
15        - allOf:
16          - allOf:
17            - path+ends: /csrutil
18          - allOf:
19            - args+contains: disable
```

Listing 21: Exemplu de euristică generată de script-ul de conversie

După apelarea scriptului de conversie, se generează euristica din fragmentul de cod 21. Numărul de puncte are o valoare prestabilită de 0 și trebuie modificat de dezvoltator în funcție de acuratețea detecției. Script-ul știe să traducă numele evenimentelor, argumentelor și comparatorilor prin consultarea unui dicționar, ce poate fi ușor adaptat pentru alte tipuri.

Interoperabilitatea cu regulile Sigma aduce un mare avantaj în momentul schimbării soluției curente de securitate, și se asigură că soluția este capabilă să detecteze evenimente pe care comunitatea de securitate cibernetică deja le folosește în scenarii reale.

3.5 Fișierul de configurare

EDR-ul are un fișier de configurare ce este citit la momentul inițializării, și descrie anumite setări inițiale. Momentan, acesta are un singur câmp, numit `useAuthEventsWhenPossible` (fragmentul de cod 22).

```
1 useAuthEventsWhenPossible: true
```

Listing 22: Fișierul de configurare

Când această valoare este falsă, EDR-ul se va abona la evenimentele de notificare, primite asincron, util în cazul unei sesiuni de debug, întrucât evenimentele de notificare nu așteaptă un răspuns și se poate pune procesul extensiei pe pauză cât se investighează problema. Valoarea setată pe adevărat configurează EDR-ul să se aboneze la evenimentele de autorizare, acolo unde există. În cazul evenimentelor ce nu au o alternativă de autorizare (e.g. `WRITE`), EDR-ul va folosi evenimentul de notificare ca soluție de rezervă. Abonarea la evenimentele de autorizare permite detecțiilor să acționeze cât mai rapid, fiind primite sincron, și să blocheze viitoare evenimente ale procesului cât timp acesta așteaptă să fie terminat de extensie.

3.6 Interfața grafică

Integratorul reprezintă interfața grafică cu care utilizatorul interacționează direct, și comunică cu extensia ES pentru a afișa detaliile relevante.

3.6.1 Integratorul

Pagina de instalare

Deoarece extensia ES poate fi instalată numai de aplicația care o conține, trebuie să existe o funcționalitate care o instalează și o dezinstalează.

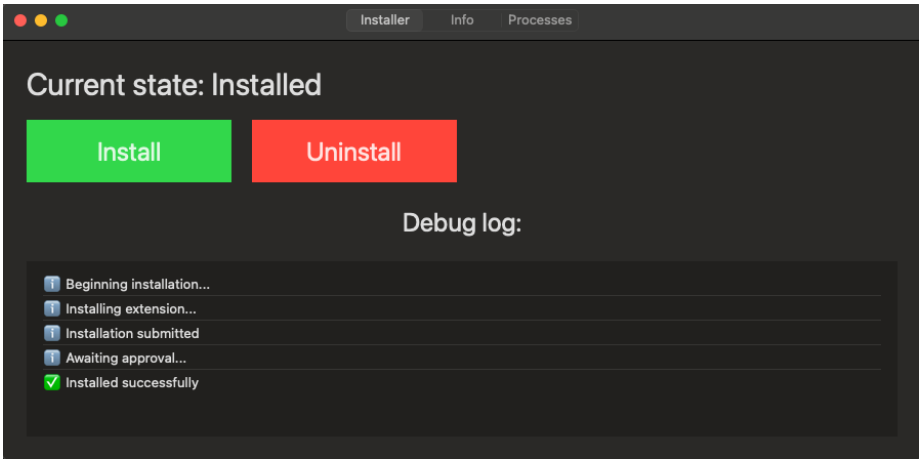


Figura 3.8: Pagina de instalare din integrator

Precum în figura 3.8, pagina afișează starea curentă a extensiei (e.g. neinstalată, în curs de aprobare, instalată), și posibilitatea de a instala și dezinstala extensia. De asemenea, afișează și o listă cu log-uri, utile în caz că există o eroare.

Pagina de informații

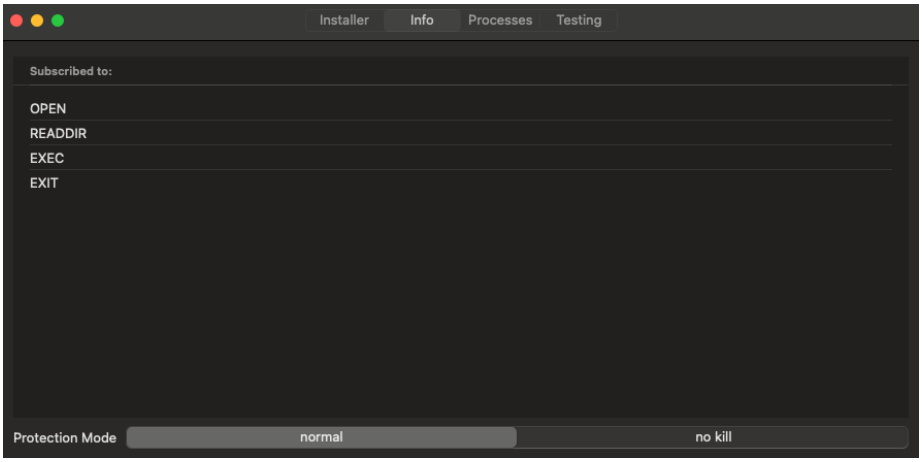


Figura 3.9: Pagina de informații din integrator

Această pagină (figura 3.9) afișează informații despre extensie, precum evenimentele la care extensia s-a abonat. De asemenea, se poate seta extensia în modul ”report-only”, în care va evita terminarea proceselor chiar dacă au depășit numărul maxim de puncte, această funcționalitate fiind utilă pentru testare.

Pagina de procese

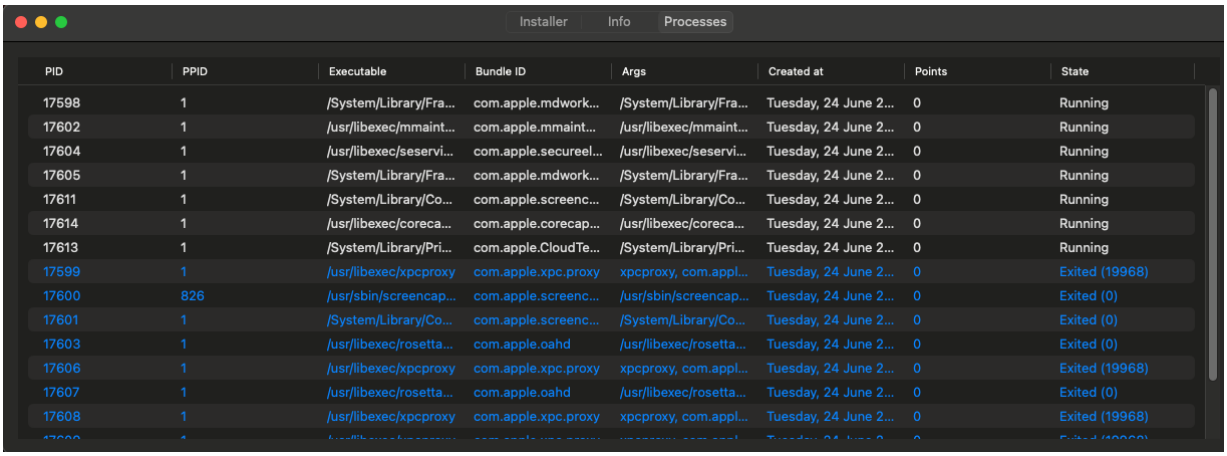


Figura 3.10: Pagina de procese din integrator

Pagina de procese (figura 3.10) afișează toate procesele din sistem și informații relevante despre acestea. Pe măsură ce extensia primește evenimente de la subsistemul ES, pagina este actualizată pentru a reflecta modificările (e.g. crearea unui proces, terminarea

unui proces, asocierea unei noi euristici). Informațiile afișate în tabel despre fiecare proces sunt:

- **PID** - identificatorul procesului care rulează
- **PPID** - identificatorul procesului părinte
- **Calea executabilului** - locația pe disk a binarului
- **Bundle ID** - identificatorul unic al aplicației
- **Argumentele** - lista de argumente trimise la linia de comandă
- **Timpul** - momentul când procesul a fost deschis
- **Numărul de puncte** - suma tuturor punctelor euristicilor asociate procesului
- **Starea procesului**
  - **running** - procesul rulează
  - **exited** - procesul a ieșit fără interacțiunea extensiei
  - **awaiting termination** - procesul a depășit limita de puncte, iar extensia este pe cale sa îl oprească
  - **terminated** - procesul a fost oprit de extensie

Procesele sunt grupate după relația părinte-copil, și în funcție de numărul de puncte. De asemenea, procesele care au ieșit sunt afișate la final.

Când o euristică este asociată unui proces, pagina prioritizează procesul afectat și îl colorează cu galben. Toate euristicile aplicate se pot vedea apăsând pe un proces. (figura 3.11).

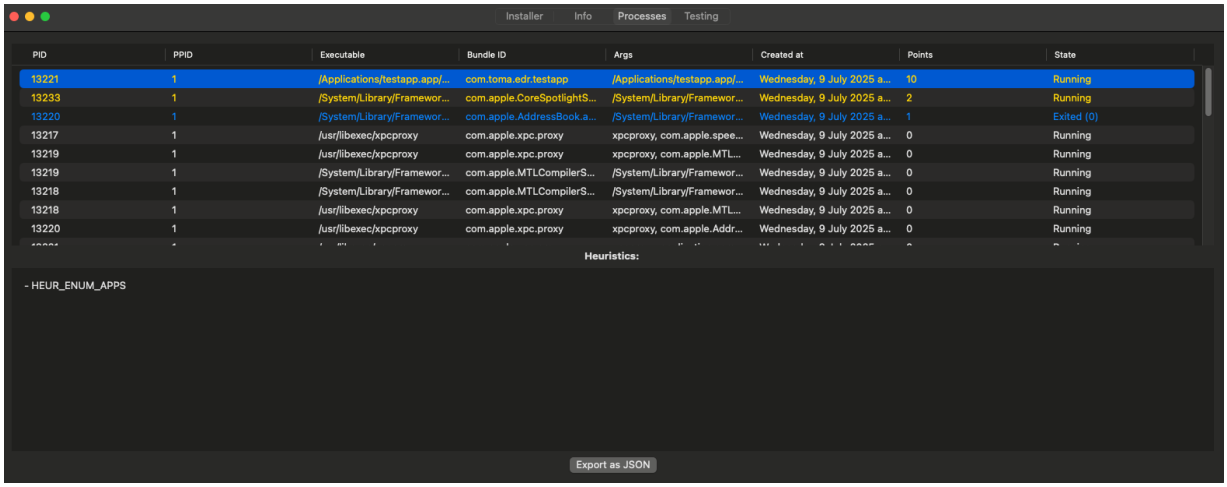


Figura 3.11: Exemplu de proces care are asociată cel puțin o euristică

Atunci când destule euristici adună puncte, iar procesul trece de un număr prestabilit (50), procesul este oprit, iar intergratorul este notificat, care va afișa procesul cu roșu (figura 3.12).

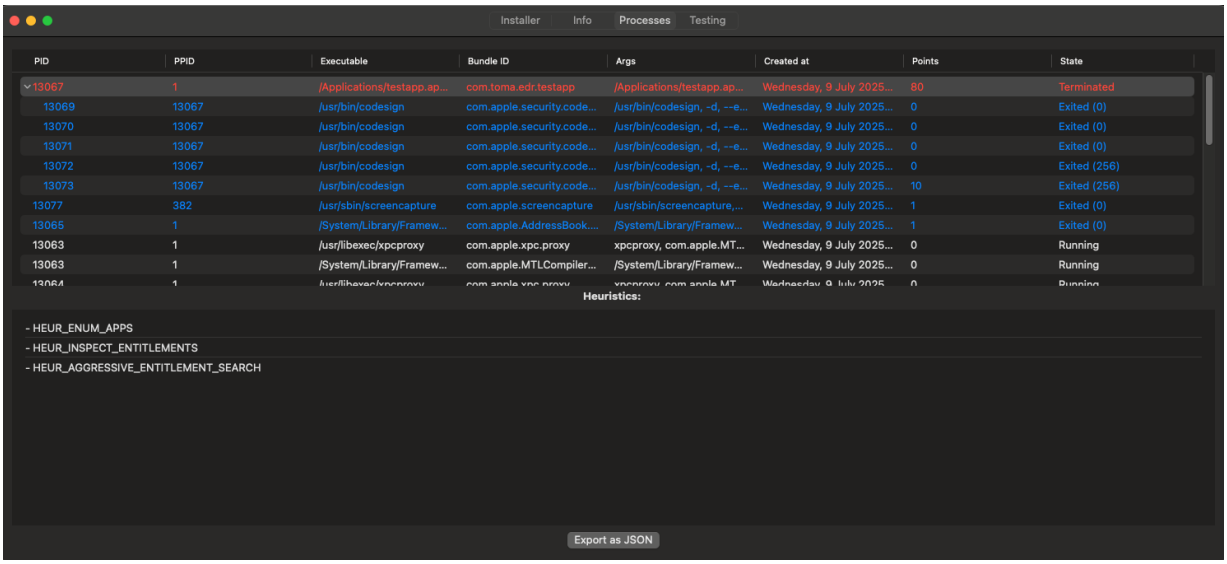


Figura 3.12: Exemplu de proces care depășit numărul maxim de puncte și a fost terminat

Procesele sunt colorate după următoarea legendă:

- **albastru** - procesul a ieșit
- **galben** - are mai mult de 0 puncte
- **portocaliu** - a trecut de 50 de puncte și așteaptă să fie închis de extensie
- **roșu** - închis de extensie
- **alb** - procesul rulează și nu are nimic ieșit din comun

De asemenea, în partea de jos a paginii se află butonul `Export as JSON`, ce salvează informațiile despre procese într-un fișier local, pentru a putea fi folosit în script-uri.

Pagina de testare

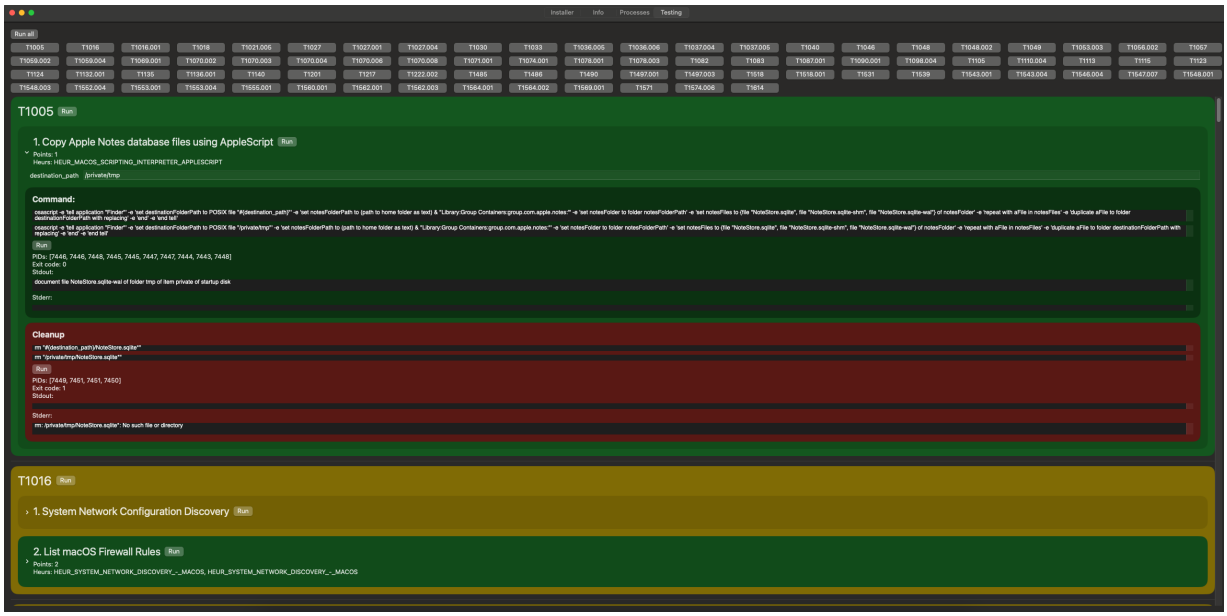


Figura 3.13: Pagina de testare din integrator

În figura 3.13 se află pagina de testare din aplicație, ce rulează comenzi din suita de teste Atomic Red Team [21]. Interfața grupează testele după tehnici Mitre și permite rularea comenzilor individuale, dar și a testelor și a întregilor tehnici.

În partea de sus se află toate tehnicile Mitre ce au cel puțin un test valid pentru macOS în suita Atomic, iar fiecare buton va aduce în prim plan testele din lista glisantă.

Fiecare tehnică, test și comandă (inclusiv dependințele, în cazul anumitor teste) sunt colorate folosind următoarea legendă pentru comenzi:

- **verde**
  - **comenzile de test** - testul a trecut și există cel puțin o euristică
  - **celelalte comenzi** - comanda a ieșit cu codul 0
- **roșu**
  - **comenzile de test** - testul s-a finalizat, dar fără nicio euristică validă
  - **celelalte comenzi** - comanda a ieșit cu un cod diferit de 0
- **galben** - comanda nu a fost executată încă

În cazul testelor și tehnicilor, culorile sunt similare, dar determină succesul sau eșecul întregului grup.

3.6.2 Aplicația de testare

Pentru a testa detecția unui EDR, este important să avem o aplicație ce poate simula comportament rău intenționat, dar care să nu dăuneze sistemului, cum ar face un malware. Pentru testarea soluției prezentate, a fost folosită o aplicație ce poate genera cele 3 euristici prezentate anterior. Interfața este simplă, cu un buton pentru fiecare tip de euristică și un câmp unde se pot scrie rezultatul acțiunilor (figura 3.14). Aplicația poate fi ușor extinsă pentru a acomoda orice tip de euristică care este adăugată în sistem. Este scrisă în Swift, pentru a arăta că, deși nu folosim direct apelurile de sistem, limbajele "high-level" care expun funcții utilitare pentru diverse acțiuni ajung să apeleze kernel-ul, generând evenimente ES.

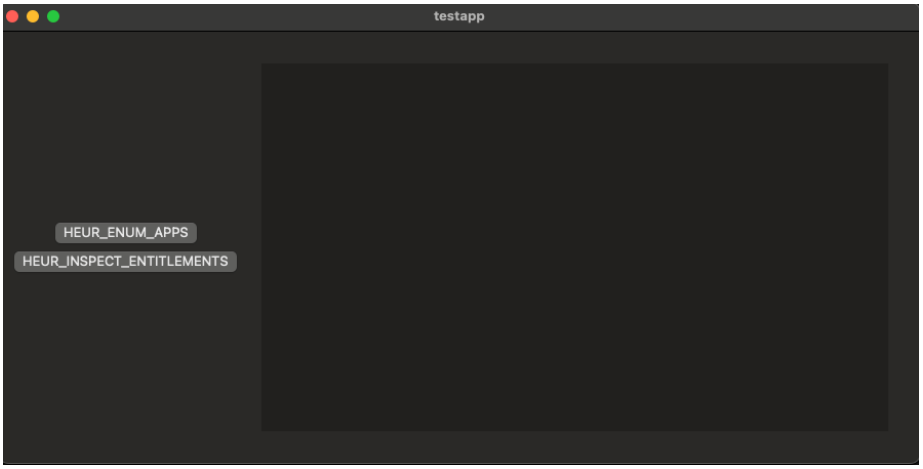


Figura 3.14: Aplicația de testare

HEUR_ENUM_APPS

Această euristică enumeră toate aplicațiile din folderul `/Applications` prin apelarea funcțiilor din clasa `FileManager` (fragmentul de cod 23).

```
1 func enumFolder(name: String) -> [String] {
2     try! FileManager.default.contentsOfDirectory(atPath: name)
3 }
```

Listing 23: Enumerarea unui folder în Swift

După apăsarea butonului, secțiunea din dreapta conține numele tuturor intrărilor (figura 3.15), iar pagina de procese a integratorului va afișa noua euristică și numărul de puncte (figura 3.11).

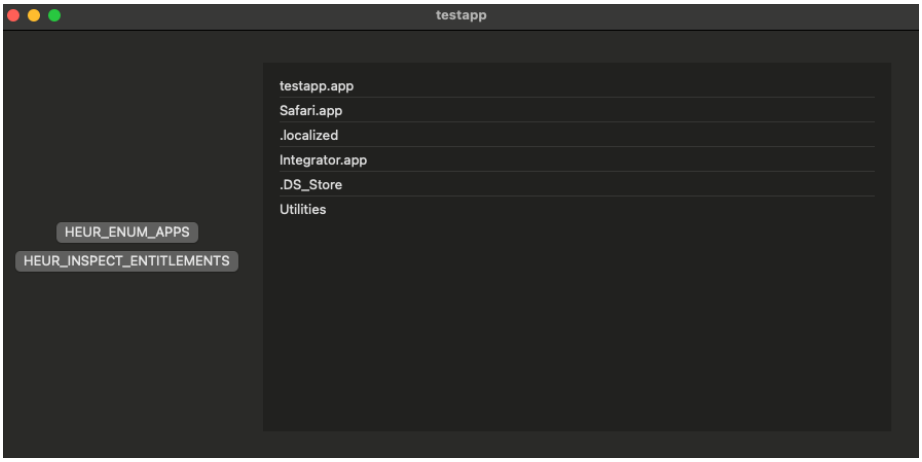


Figura 3.15: Rezultatul enumerării folderului

HEUR_INSPECT_ENTITLEMENTS

Această euristică descrie acțiunea de a inspecta entitlement-urile unei aplicații. Acest lucru se face prin apelarea binarului preinstalat `codesign` cu argumentul `--entitlements`, iar în Swift se poate obține similar fragmentului de cod 24.

```
1  func getEntitlements(app: String) -> String {
2      let task = Process()
3      task.executableURL = URL(fileURLWithPath: "/usr/bin/codesign")
4      task.arguments = ["-d", "--entitlements", "-", "--xml",
       ↪  "/Applications/\(app)"]
5      let stdout = Pipe()
6      task.standardOutput = stdout
7      try! task.run()
8      let outputData = stdout.fileHandleForReading.availableData
9      return String(decoding: outputData, as: UTF8.self)
10 }
```

Listing 24: Inspectarea entitlement-urilor unei aplicații în Swift

HEUR_AGGRESSIVE_ENTITLEMENT_SEARCH

Dacă EDR-ul este activ, va observa că cele două euristici au fost generate, și va genera și a treia euristică (`HEUR_AGGRESSIVE_ENTITLEMENT_SEARCH`), care va opri procesul, întrucât va depăși numărul de puncte. În schimb, dacă EDR-ul este dezactivat, sau euristicile modificate, aplicația de test va actualiza panoul cu informații despre entitlements (figura 3.16).



Figura 3.16: Rezultatul enumerării entitlement-urilor

Capitolul 4

Rezultate teoretice și experimentale

4.1 Suita de teste Atomic Red Team

În cadrul testelor de detecție, s-au folosit testele din suita Atomic Red Team [21] și regulile Sigma [20] din care s-au generat euristici folosind script-ul de conversie.

În tabelul 4.1 se află rezultatele testelor grupate după tehnicile Mitre. În cazul în care o tehnică are mai multe teste, ele sunt descrise folosind un index în paranteze rotunde. Convenția folosită pentru a colora celulele tabelului este următoarea:

- **verde** - cel puțin o euristică validă
- **roșu** - nicio euristică
- **galben** - există cel puțin o euristică, dar nu este destul de specifică (e.g. o comandă rulată prin AppleScript din care reiese o euristică generală de rulare a script-urilor AppleScript, nu una specifică pentru acțiunea efectuată de script)

T1005	T1016(1)	T1016(2)	T1016.001	T1018(1)
T1018(2)	T1021.005	T1027	T1027.001(1)	T1027.001(2)
T1027.004(1)	T1027.004(2)	T1027.004(3)	T1030	T1033
T1036.005	T1036.006	T1037.004	T1037.005(1)	T1037.005(2)
T1037.005(3)	T1040(1)	T1040(2)	T1040(3)	T1046(1)
T1046(2)	T1046(3)	T1048(1)	T1048(2)	T1048(3)
T1048.002	T1049	T1053.003	T1056.002(1)	T1056.002(2)
T1057	T1059.002	T1059.004(1)	T1059.004(2)	T1059.004(3)
T1069.001	T1070.002(1)	T1070.002(2)	T1070.002(3)	T1070.002(4)
T1070.002(5)	T1070.002(6)	T1070.002(7)	T1070.002(8)	T1070.002(9)
T1070.002(10)	T1070.002(11)	T1070.003(1)	T1070.003(2)	T1070.003(3)
T1070.003(4)	T1070.003(5)	T1070.003(6)	T1070.004(1)	T1070.004(2)

T1070.006(1)	T1070.006(2)	T1070.006(3)	T1070.006(4)	T1070.006(5)
T1070.008(1)	T1070.008(2)	T1071.001	T1074.001	T1078.001
T1078.003(1)	T1078.003(2)	T1078.003(3)	T1082(1)	T1082(2)
T1082(3)	T1082(4)	T1082(5)	T1082(6)	T1083(1)
T1083(2)	T1087.001(1)	T1087.001(2)	T1087.001(3)	T1087.001(4)
T1087.001(5)	T1090.001(1)	T1090.001(2)	T1098.004	T1105(1)
T1105(2)	T1105(3)	T1105(4)	T1105(5)	T1105(6)
T1105(7)	T1105(8)	T1110.004	T1113(1)	T1113(2)
T1115	T1123	T1124	T1132.001	T1135
T1136.001	T1140(1)	T1140(2)	T1140(3)	T1140(4)
T1140(5)	T1140(6)	T1201	T1217(1)	T1217(2)
T1217(3)	T1222.002(1)	T1222.002(2)	T1222.002(3)	T1222.002(4)
T1222.002(5)	T1222.002(6)	T1222.002(7)	T1222.002(8)	T1222.002(9)
T1222.002(10)	T1222.002(11)	T1485	T1486(1)	T1486(2)
T1490	T1497.001(1)	T1497.001(2)	T1497.001(3)	T1497.001(4)
T1497.003	T1518	T1518.001	T1531(1)	T1531(2)
T1531(3)	T1539	T1543.001(1)	T1543.001(2)	T1543.004
T1546.004(1)	T1546.004(2)	T1547.007(1)	T1547.007(2)	T1547.007(3)
T1548.001(1)	T1548.001(2)	T1548.001(3)	T1548.003(1)	T1548.003(2)
T1552.004(1)	T1552.004(2)	T1552.004(3)	T1553.001	T1553.004
T1555.001(1)	T1555.001(2)	T1555.001(3)	T1555.001(4)	T1560.001(1)
T1560.001(2)	T1560.001(3)	T1560.001(4)	T1562.001(1)	T1562.001(2)
T1562.001(3)	T1562.001(4)	T1562.001(5)	T1562.003	T1564.001(1)
T1564.001(2)	T1564.001(3)	T1564.001(4)	T1564.001(5)	T1564.002(1)
T1564.002(2)	T1569.001	T1571	T1574.006	T1614

Tabela 4.1: Rezultatele testelor Atomic folosind doar regulile Sigma

Din cele 185 de teste, rezultatele au fost următoarele:

- **verde** - 75/185 (40.54%)
- **roșu** - 105/185 (56.75%)
- **galben** - 5/185 (2.79%)

Rezultatul poate fi, la prima vedere, unul dezamăgitor, însă trebuie să avem în vedere următoarele aspecte:

- Euristicile au fost generate din cele Sigma, care sunt scrise de comunitate, iar cele de pe macOS nu sunt așa întreținute precum cele de pe Windows sau Linux; numărul de euristici de pe fiecare platformă este:

  - **Windows** - 2301
  - **Linux** - 205
  - **macOS** - 68

  ceea ce arată că există de 3 ori mai multe reguli Sigma pentru Linux, și de 33 de ori mai multe pentru Windows

- Multe teste din suita Atomic sunt comune cu macOS și Linux, dar regulile Sigma echivalente de pe Linux nu sunt traduse și pentru macOS

- Regulile Sigma pe macOS au doar 2 categorii (creare de proces și acțiune pe fișiere), în timp ce celelalte platforme au mult mai multe, iar unele teste din suita Atomic au nevoie de evenimente mult mai specifice

Însă, EDR-ul este foarte ușor de extins și putem adăuga euristici noi (chiar și cu alte tipuri de evenimente) pentru a detecta majoritatea testelor care nu au fost detectate doar cu regulile Sigma.

În tabelul 4.2 sunt afișate rezultatele testelor după ce au fost adăugate mai multe euristici. Convenția culorilor celulelor este aceeași ca în tabelul precedent.

T1005	T1016(1)	T1016(2)	T1016.001	T1018(1)
T1018(2)	T1021.005	T1027	T1027.001(1)	T1027.001(2)
T1027.004(1)	T1027.004(2)	T1027.004(3)	T1030	T1033
T1036.005	T1036.006	T1037.004	T1037.005(1)	T1037.005(2)
T1037.005(3)	T1040(1)	T1040(2)	T1040(3)	T1046(1)
T1046(2)	T1046(3)	T1048(1)	T1048(2)	T1048(3)
T1048.002	T1049	T1053.003	T1056.002(1)	T1056.002(2)
T1057	T1059.002	T1059.004(1)	T1059.004(2)	T1059.004(3)
T1069.001	T1070.002(1)	T1070.002(2)	T1070.002(3)	T1070.002(4)
T1070.002(5)	T1070.002(6)	T1070.002(7)	T1070.002(8)	T1070.002(9)
T1070.002(10)	T1070.002(11)	T1070.003(1)	T1070.003(2)	T1070.003(3)
T1070.003(4)	T1070.003(5)	T1070.003(6)	T1070.004(1)	T1070.004(2)
T1070.006(1)	T1070.006(2)	T1070.006(3)	T1070.006(4)	T1070.006(5)
T1070.008(1)	T1070.008(2)	T1071.001	T1074.001	T1078.001
T1078.003(1)	T1078.003(2)	T1078.003(3)	T1082(1)	T1082(2)
T1082(3)	T1082(4)	T1082(5)	T1082(6)	T1083(1)
T1083(2)	T1087.001(1)	T1087.001(2)	T1087.001(3)	T1087.001(4)
T1087.001(5)	T1090.001(1)	T1090.001(2)	T1098.004	T1105(1)
T1105(2)	T1105(3)	T1105(4)	T1105(5)	T1105(6)
T1105(7)	T1105(8)	T1110.004	T1113(1)	T1113(2)
T1115	T1123	T1124	T1132.001	T1135
T1136.001	T1140(1)	T1140(2)	T1140(3)	T1140(4)

T1140(5)	T1140(6)	T1201	T1217(1)	T1217(2)
T1217(3)	T1222.002(1)	T1222.002(2)	T1222.002(3)	T1222.002(4)
T1222.002(5)	T1222.002(6)	T1222.002(7)	T1222.002(8)	T1222.002(9)
T1222.002(10)	T1222.002(11)	T1485	T1486(1)	T1486(2)
T1490	T1497.001(1)	T1497.001(2)	T1497.001(3)	T1497.001(4)
T1497.003	T1518	T1518.001	T1531(1)	T1531(2)
T1531(3)	T1539	T1543.001(1)	T1543.001(2)	T1543.004
T1546.004(1)	T1546.004(2)	T1547.007(1)	T1547.007(2)	T1547.007(3)
T1548.001(1)	T1548.001(2)	T1548.001(3)	T1548.003(1)	T1548.003(2)
T1552.004(1)	T1552.004(2)	T1552.004(3)	T1553.001	T1553.004
T1555.001(1)	T1555.001(2)	T1555.001(3)	T1555.001(4)	T1560.001(1)
T1560.001(2)	T1560.001(3)	T1560.001(4)	T1562.001(1)	T1562.001(2)
T1562.001(3)	T1562.001(4)	T1562.001(5)	T1562.003	T1564.001(1)
T1564.001(2)	T1564.001(3)	T1564.001(4)	T1564.001(5)	T1564.002(1)
T1564.002(2)	T1569.001	T1571	T1574.006	T1614

Tabela 4.2: Rezultatele testelor Atomic folosind reguli Sigma și euristici personalizate

După adăugarea noilor euristici, avem următoarele statistici:

- **verde** - 176/185 (95.13%)
- **roșu** - 8/185 (4.32%)
- **galben** - 1/185 (0.54%)

Diferențele sunt semnificative, ajungând ca peste 95% din testele Atomic să fie detectate folosind EDR-ul. Pentru restul de teste nu a fost găsită o euristică potrivită din motive variate:

- Testul apelează funcții legate de rețea (e.g. capturarea pachetelor); ES nu are evenimente specifice pentru astfel de scenarii, pentru că există un alt SDK specific pentru rețea numit `NetworkExtension`
- Testul folosește funcții specifice limbajelor interpretate (e.g. funcții din Python, variabile din Bash); în timp ce, teoretic, se poate citi fișierul trimis la interpretor, acest lucru poate aduce următoarele dezavantaje:
  - impact de performanță - citirea unui fișier întreg de fiecare dată când este accesat este o operație costisitoare; această acțiune este mai potrivită pentru un sistem anti-malware
  - comenzile pot fi obfuscate și imposibil de parsat

Capitolul 5

Concluzii

Teza prezentată a urmărit dezvoltarea și detalierea funcționalităților finale a unui client Apple Endpoint Security, cu scopul de a detecta și bloca atacuri din surse rău intenționate folosind un sistem de detecție comportamentală. La începutul tezei a fost setată o listă de obiective, care urmează a fi explorate dacă au fost atinse sau nu:

- **Investigarea atacurilor recente și notarea metodelor folosite de atacatori**: Au fost studiate diverse articole și postări despre comportamentul programelor rău intenționate și evoluția acestora în ultimii ani. A fost făcută o corelare între acțiunile lor și potențiale evenimente din sistemul ES, precum și alte metode de a le bloca.

- **Implementarea unui client ES capabil să primească evenimente relevante din sistem și filtrarea acestora în funcție de proces**: A fost descris un sistem de detecție comportamentală ce folosește evenimente relevante provenite din sistemul ES, ce sunt trecute printr-un motor de euristici ce transformă o acțiune rău intenționată într-un șir de evenimente cărora le sunt aplicate condiții

- **Scrierea unor tehnici adecvate care pot detecta și bloca atacurile înainte să aibă vreun impact major**: Au fost scrise diverse euristici ce arată flexibilitatea sistemului și a fost dezvoltat un script de conversie capabil de a transforma reguli Sigma în euristici compatibile.

- **Testarea întregii soluții și evaluarea eficienței asupra atacurilor moderne**: A fost rulată întreaga suită de teste Atomic pentru macOS, prima dată doar cu euristicile transformate din regulile Sigma, și a doua oară când au fost adăugate reguli personalizate. Rezultatul final este unul deosebit de bun, soluția reușind să detecteze mai mult de 95% din teste.

În urma analizei, toate obiectivele propuse au fost atinse. Soluția prezentată reprezintă o bază solidă pentru un EDR performant și modern, capabil să detecteze atacuri deja existente, dar și atacuri necunoscute, având în vedere granularitatea sistemului de euristici și suita bogată de evenimente oferite de Endpoint Security.

Bibliografie

[1] A. N. Halina Kowalczyk, Piotr Zieliński, "Dissecting macos ransomware: A comparative analysis and mitigation strategies," https://www.researchsquare.com/article/rs-4385485/v1, 2024.

[2] F. M. Duy-Phuc Pham 1, Duc-Ly Vu2, "Mac-a-mal: macos malware analysis framework resistant to antievasion techniques," *Journal of Computer Virology and Hacking Techniques*, 2019.

[3] M. D. Nguyen, "A poisoned apple: The analysis of macos malware shlayer," https://www.cs.tufts.edu/comp/116/archive/spring2018/mnguyen.pdf, 2018.

[4] S. by Kaspersky, "Shlayer trojan attacks one in ten macos users," https://securelist.com/shlayer-for-macos/95724/, 2020.

[5] C. Fitzl, "Mitigating exploits using apple's endpoint security," https://theevilbit.github.io/talks_workshops/2021/VirusBulletin2021-Fitzl-mitigating-exploits-using-Apple-es-WP.pdf, 2021.

[6] Elastic, "Mac system extensions for threat detection: Part 1," https://www.elastic.co/blog/mac-system-extensions-for-threat-detection-part-1.

[7] Google, "Santa," https://github.com/google/santa.

[8] northpolesec, "Santa," https://github.com/northpolesec/santa.

[9] Google, "Santa documentation," https://santa.dev.

[10] SuprHackerSteve, "Crescendo," https://github.com/SuprHackerSteve/Crescendo.

[11] theevilbit, "Shield - an app to protect against process injection on macos," https://theevilbit.github.io/shield/.

[12] Objective-See, "Writing a process monitor with apple's endpoint security framework," https://objective-see.org/blog/blog_0x47.html.

[13] ——, "Writing a file monitor with apple's endpoint security framework," https://objective-see.org/blog/blog_0x48.html.

[14] Apple, “Endpoint security,” https://developer.apple.com/documentation/endpointsecurity.

[15] ——, “Build an endpoint security app - wwdc20,” https://developer.apple.com/videos/play/wwdc2020/10159/.

[16] ——, “Monitoring system events with endpoint security,” https://developer.apple.com/documentation/endpointsecurity/monitoring-system-events-with-endpoint-security.

[17] ——, “Swift,” https://developer.apple.com/swift/.

[18] ——, “Swiftui,” https://developer.apple.com/documentation/swiftui/.

[19] ——, “Xpc,” https://developer.apple.com/documentation/xpc.

[20] SigmaHQ, “Sigma,” https://github.com/SigmaHQ/sigma.

[21] redcanaryco, “Atomic red team,” https://github.com/redcanaryco/atomic-red-team.